

第5章: React・フロントエンド詳細 — 画面の組み立て方（完全版）

この章では、本アプリの画面を作っている技術「**React（リアクト）**」を、**本アプリの実際のコンポーネント `KosuList.tsx` 全215行を頭から最後まで1行ずつ** 読み解きながら学びます。第3章で「HTMLとCSS（見た目の骨組み）」を、第4章で「JavaScript／TypeScript（動きの言語）」を学びました。この第5章は、その2つを **部品（コンポーネント）** という単位で組み立て、**データに応じて画面を自動で書き換える** Reactの世界です。

Reactは「最初の山」です。 `useState` ・ `useEffect` ・ `useCallback` ・ `useRef` ・ `props` ・ `JSX` ……カタカナと記号が一気に押し寄せます。でも安心してください。この章は **本アプリの実物コードだけ** を題材に、たとえ話を山ほど使って、1行も飛ばさずに説明します。読み終えたとき、あなたは「工数一覧画面（`KosuList`）が、なぜ・どうやって動いているか」を完全に説明できるようになります。

この章も長いですが、**辞書のように使えること** を目指しています。「`useEffect` の依存配列って何だっけ」と思ったら、ここに返って該当箇所を引いてください。

この章で学ぶこと

- **Reactとは何か** — 「部品（コンポーネント）を組み立てて画面を作る」という考え方を、家・レゴ・料理でたとえる
- **コンポーネント** — 画面の部品。本アプリの `KosuList` ・ `Pagination` ・ `Loading` ・ `TableContainer` が実例
- **JSX** — JavaScriptの中にHTMLを書く記法（第3章の復習＋深掘り）
- **props（プロップス）** — 親から子へ渡す「設定値」。 `Pagination` への受け渡しを実コードで追う
- **state（ステート）と `useState`** — 画面が覚えておく「変化する値」。再描画のしくみ
- **`useEffect`** — 「あるタイミングで処理を走らせる」しくみ。本アプリの2つの `useEffect` を完全解説
- **`useRef`** — 再描画を起こさずに値や要素を覚えておく箱
- **`useCallback`** — 関数を「使い回す」最適化。 `fetchData` で実例

- **KosuList.tsx** 全215行を1行ずつ — import群／interface／変換関数／全state／fetchData／2つのuseEffect／handleSearch／早期リターン／JSX（nav・検索バー・table・map・Pagination）
- **Pagination.tsx** 全71行を1行ずつ — propsの受け渡しとデフォルト値、CSS変数の注入
- **Loading.tsx** ・ **TableContainer.tsx** — children・クリーンアップ・リサイズ対応
- **axios.ts** 全8行 — サーバーと話すための共通設定（baseUrl・Cookie・CSRF）
- **index.tsx** のルーティング — **Routes** ／ **Route** ／ **:id** ／ **Link** ／ **useNavigate** ／SPA
- **トラブルシューティング・演習問題**

目次

0. 前提知識：Reactが解決した問題
 1. コンポーネントという考え方
 2. JSX — JavaScriptの中のHTML
 3. props — 親から子へ渡す設定値
 4. state と useState — 画面が覚える値
 5. useEffect — タイミングで処理を走らせる
 6. useRef — 再描画を起こさない箱
 7. useCallback — 関数を使い回す
 8. 実コード全解説：KosuList.tsx 全215行
 9. 実コード全解説：Pagination.tsx 全71行
 10. 実コード全解説：Loading.tsx 全42行
 11. 実コード全解説：TableContainer.tsx 全57行
 12. 実コード全解説：axios.ts 全8行
 13. ルーティング：index.tsx 完全解説とSPA
 14. Link と useNavigate — 画面の移動
 15. 描画サイクルの全体像（まとめ図）
 16. トラブルシューティング
 17. 演習問題
 18. この章のまとめ
-

0. 前提知識：Reactが解決した問題

「なぜReactが必要なのか」を最初に腹落ちさせておきます。これが分かったら、以降の `useState` や `useEffect` の「ありがたみ」が理解できます。

0.1 Reactがなかった時代の苦勞

第3章で「JavaScriptがDOMを書き換えると画面が更新される」と学びました。たとえば「ボタンを押したら一覧に1行追加する」を、Reactなしの素のJavaScriptで書くと、こんなことを **手作業** でやる必要がありました。

※説明用の簡易例（本アプリのコードではありません）

```
// ※説明用の簡易例：素のJavaScriptで1行追加する
const tr = document.createElement("tr");           // 行（tr要素）を手で作る
const td = document.createElement("td");           // セル（td要素）を手で作る
td.textContent = "2025-06-01";                     // セルに文字を入れる
tr.appendChild(td);                                // 行にセルを差し込む
document.getElementById("tbody").appendChild(tr);  // 表の本体に行を差し込む
```

データが1つ増えるたびに、「**どのDOMを、どこに、どう差し込むか**」を**全部自分で指示**していました。データが100個・1000個と増え、削除や並べ替えが入ると、この手作業はすぐに破綻します。「画面とデータがズレる（表示は古いままなのに裏のデータは新しい）」というバグの温床でした。

0.2 Reactの発想 — 「データを変えたら、画面は勝手に作り直す」

Reactの発想は革命的でした。

Reactの中心思想: プログラマは「**今のデータなら、画面はこうあるべき**」という **設計図** だけを書く。データが変わったら、Reactが **古い画面と新しい設計図を比べて、変わった所だけを自動で書き換える**。

料理でたとえます。古いやり方は「冷蔵庫の中身が変わるたびに、料理人が手で皿を1枚ずつ並べ替える」。Reactは「**献立表（設計図）を渡せば、シェフ（React）が今ある食材で最適に盛り付け直してくれる**」。私たちは献立表（=どんなデータならどんな画面か）だけ考えればよい——これが革命でした。

用語: 宣言的（せんげんてき／declarative） 「どうやるか（手順）」ではなく「どうあるべきか（結果の姿）」を書くスタイル。Reactは宣言的。逆に手順を1つずつ書くのが「命令的（めいれいてき／imperative）」。§ 0.1の素のJSは命令的でした。

0.3 仮想DOM — 「下書きで比べてから清書する」

Reactが「変わった所だけ書き換える」をどう実現しているか。鍵は **仮想DOM（かそうディーオーエム）** です。

用語: 仮想DOM（Virtual DOM） 本物のDOM（画面）を、JavaScriptのオブジェクト（軽い「下書き」）としてメモリ上にコピーしたもの。Reactは「新しい下書き」と「古い下書き」を比較し、**違う部分だけ**を本物のDOMに反映する。本物のDOM操作は重いので、下書きで先に差分を計算することで高速になる。

漫画家でたとえると、いきなりペン入れ（本物のDOM操作=重い）をするのではなく、**鉛筆の下書き（仮想DOM=軽い）**を2枚並べて「このコマだけ違う」と見つけ、その1コマだけペン入れする、というイメージです。

0.4 本アプリにおけるReactの位置づけ

種類	役割	本アプリの該当
HTML（JSX）	画面の構造	<code>.tsx</code> の <code>return (...)</code> の中
CSS	見た目	<code>.module.css</code> / <code>global.css</code>
React + TypeScript	部品化・データ連動・動き	<code>.tsx</code> の処理部分（ <code>useState</code> 等）

この章は3つ目の「React + TypeScript」に集中します。次節から、その中身を1つずつ分解していきます。

1. コンポーネントという考え方

1.1 コンポーネント = 画面の「部品」

用語: コンポーネント (component : 部品・構成要素) 画面を構成する「再利用できる部品」のこと。Reactでは、**1つの関数が1つの部品** になります。その関数は「画面の一部 (JSX)」を返します。

レゴでたとえます。レゴの完成品（お城）は、小さなブロック（窓パーツ・ドアパーツ・壁パーツ）の組み合わせです。Reactも同じで、本アプリの画面は次のような部品の組み合わせです。

```
KosuList (工数一覧画面ぜんたい)      ← 大きな部品
├─ nav (メニューリンク)
├─ search-bar (日付検索バー)
├─ TableContainer (スクロールする表の枠) ← 中くらいの部品
│   └─ table (表本体)
│   └─ Pagination (ページ送りボタン)    ← 小さな部品 (使い回せる)
└─ Loading (読み込み中のヘルメット)     ← 小さな部品 (使い回せる)
```

`Pagination` (ページ送り) や `Loading` (読み込み中表示) は、工数一覧だけでなく **人員一覧・班員一覧など、本アプリの多くの画面で同じ部品を使い回し** ています。これがコンポーネントの最大の利点です。「1度作れば、どこでも使える」。

1.2 コンポーネントの最小の形

本アプリの実コードで、最小のコンポーネントの形を確認します。`Login.tsx` の骨格 (中身を省略) はこうです。

▼ **このコードがやること (先に日本語で)** : `Login` という名前の「関数コンポーネント」を定義しています。この関数は画面 (JSX) を `return` で返します。最後の `export default` で「この部品を他のファイルから使えるように公開」しています。

```
const Login: React.FC = () => { // Login という名前の部品を定義。React.FC は「Reactの関
// ... (中で値を準備する処理) ...
  return ( // この部品が描く画面 (JSX) を返す
    <div>...</div>
  );
};

export default Login; // この部品を外部から import できるよう公開 (既定の輸出
```

- `const Login = () => { ... }` … 第4章で学んだ **アロー関数**。これが「部品の本体」です。
- `: React.FC` … TypeScriptの型注釈。「これはReactの **F**unction **C**omponent (関数コンポーネント) です」という宣言。
- `return ( <div>...</div> )` … この部品が画面に出す中身。`( )` で囲むのは「複数行のJSXを返すときの作法」。
- `export default Login` … 第4章で学んだ **default export**。他のファイルが `import Login from "./Login"` で取り込みます。

用語: React.FC (エフシー) `FunctionComponent` の略。「引数に props を受け取り、JSXを返す関数」であることをTypeScriptに伝える型。付けておくと、props の型チェックや補完が効きます。

⚠ **コンポーネント名は必ず大文字始まり**。 `Login` ・ `KosuList` ・ `Pagination` のように先頭を大文字にします。 `login` と小文字で始めると、Reactは「ただのHTMLタグ」と勘違いします (`<login>` は存在しないタグなので何も出ません)。これは初心者が必ず1度はハマる落とし穴です。

1.3 部品を「使う」とは 一タグのように書く

定義した部品は、JSXの中で **HTMLタグのように** 書いて使います。 `KosuList.tsx` の中では、 `Pagination` 部品をこう使っています (詳細は §8 ・ §9)。

```
<Pagination
  currentPage={currentPage}
  totalPages={totalPages}
  setCurrentPage={setCurrentPage}
  buttonColor="#0ff"
  hoverColor="#0af"
/>
```

`<Pagination ... />` と、まるで自作のHTMLタグのように書けます。`currentPage={...}` などは、後で説明する **props（親から子へ渡す設定値）** です。「ページ送り部品を、この設定で1つ置いてね」という指示になります。

2. JSX — JavaScriptの中のHTML

第3章で軽く触れたJSXを、Reactの視点で深掘りします。

用語: JSX（ジェイエスエックス） JavaScript（TypeScript）の中に、HTMLのような記述を直接書ける拡張記法。Reactで画面を書くための標準的な書き方。ファイルの拡張子が `.tsx` なのは「TypeScript + JSX」の意味です。

2.1 HTMLとの3つの違い（重要）

JSXはHTMLによく似ていますが、JavaScriptの中を書く都合上、3つの違いがあります。

HTML	JSX	理由
<code>class="..."</code>	<code>className="..."</code>	<code>class</code> はJavaScriptの予約語（クラス定義に使う）なので別名に
<code>for="..."</code>	<code>htmlFor="..."</code>	<code>for</code> もJavaScriptの予約語（forループ）なので別名に
値は文字列だけ	<code>{ }</code> でJSの値を埋め込める	JSXはJavaScriptの一部だから

`Login.tsx` の実例で確認します。

```
<label htmlFor="numberInput">従業員番号</label>          {/* for ではなく htmlFor */}
<input id="numberInput" className={styles["input-focus"]} /> {/* class ではなく className
```

2.2 `{ }` — 中括弧でJavaScriptを埋め込む

JSXの中で `{ }` を書くと、その中は **JavaScriptの世界** になります。変数・計算・関数呼び出しを画面に埋め込みます。 `KosuList.tsx` の実例。

```
<td>{item.work_day2} ({getDayOfWeek(item.work_day2)})</td>
```

これは「 `item.work_day2` (就業日の文字列) を表示し、続けて括弧の中に `getDayOfWeek(...)` (曜日を返す関数) の **結果** を表示する」という意味です。たとえば `2025-06-01 (日)` のように表示されます。 `{ }` の中身は実行され、その **結果** が画面に出ます。

なぜ `{ }` なの？ HTMLはただの文字列ですが、JSXは「JavaScriptの値を画面に流し込める」のが強みです。 `{ }` が「ここからJavaScript」「ここまで」の境界線です。 `{ }` の外は普通のHTMLテキスト、中はJavaScriptと覚えてください。

2.3 条件分岐： `&&` と 三項演算子

JSXの中では `if` 文がそのままは書けません (`{ }` の中は「式」しか書けないため)。代わりに2つのテクニックを使います。

(A) **条件 `&&` <要素>** — 「条件がtrueのときだけ表示」

`Login.tsx` の実例。

```
{errorMessage && (
  <div role="alert">{errorMessage}</div>
)}
```

「 `errorMessage` が空でない (=エラーがある) のときだけ、赤いアラートを表示」。 `errorMessage` が空文字 (false扱い) なら何も表示しません。

なぜ && で表示できる？ JavaScriptの `A && B` は「Aが偽ならAを、Aが真ならBを返す」性質があります（第4章）。`errorMessage` が空なら空文字（画面に何も出ない）、中身があれば右の `<div>` が返って表示される、という仕組みです。

(B) 三項演算子 `条件 ? A : B` — 「2択」

`KosuList.tsx` の実例。

```
{data.length === 0 ? (  
  <p>No data found.</p>           // データが0件なら、このメッセージ  
) : (  
  <TableContainer>...</TableContainer> // 1件以上なら、表を表示  
)}
```

「データが0件なら『No data found.』、そうでなければ表」という2択です。`{条件 ? 真のとき : 偽のとき}` の形を覚えましょう。

2.4 リストの表示：`map`

配列の各要素を、まとめて画面に並べるには `map`（第4章）を使います。`KosuList.tsx` の核心部分。

```
{data.map((item) => (  
  <tr key={item.id}>  
    <td>{item.work_day2}</td>  
    ...  
  </tr>  
))}
```

「`data` 配列の各 `item` を `<tr>`（表の行）に変換して並べる」。データが20件あれば、20行の `<tr>` が生成されます。詳細は§8で1行ずつ解説します。

⚠ key を忘れない。 `map` で並べる各要素には `key={一意な値}`（ここでは `item.id`）が必須です。Reactが「どの行が追加・削除・移動したか」を見分ける目印です。これがないと警告が出て、再描画がおかしくなることがあります。

2.5 フラグメント `<> ... </>`

JSXは「返せるのは1つの要素だけ」というルールがあります。複数の要素を並べたいとき、無駄な `<div>` で囲む代わりに **フラグメント** `<> </>` を使います。 `KosuList.tsx` の実例。

```
return (  
  <>                                {/* フラグメント＝「見えない箱」。余計なdivを作らない */}  
    <div className={styles["kosu-list-wrapper"]} >  
      ...  
    </div>  
  </>  
);
```

用語: フラグメント (Fragment) `<>` と `</>` で囲む「見えないまとめ役」。複数要素を1つにまとめて返したいが、画面に余計な `<div>` を増やしたくないときに使う。中身だけがDOMに出力されます。

3. props — 親から子へ渡す設定値

3.1 propsとは — 「部品に渡す設定」

用語: props (プロップス／properties の略：属性・特性) 親コンポーネントから子コンポーネントへ **一方向に渡す設定値** のこと。HTMLタグの属性（ `` の `src` ）と同じ感覚で、 `<Pagination currentPage={1} />` のように渡します。

たとえ話：propsは「**部品の取扱説明書に書く設定欄**」です。エアコン（部品）を設置するとき、「設定温度=25度、風量=強」と指定しますね。Reactでは `<エアコン 温度={25} 風量="強" />` と書きます。この温度や風量が props です。

3.2 propsは「上から下」へ。逆流しない

重要なルール：**propsは親→子の一方通行** です。子が親から受け取った props を、勝手に書き換えることはできません（読み取り専用）。

```
親 (KosuList)
|   props を渡す (currentPage=1 など)
▼
子 (Pagination) ← 受け取って表示に使う。書き換えはしない
```

なぜ一方通行なの？ データの流れを1方向に固定すると、「今この値が誰のせいで変わったか」を追いやすくなり、バグが激減します。川が上流から下流へ流れるように、データも親から子へ。子が値を変えたいときは、後述する「**親から渡された関数を呼ぶ**」という方法を使います (§9で `setCurrentPage` を実演)。

3.3 propsの受け取り方 (型定義つき)

子コンポーネント側で props を受け取るには、まず **どんな props が来るか** をTypeScriptの `interface` で定義します。 `Pagination.tsx` の実コード。

```
interface PaginationProps {           // この部品が受け取る props の「設計図」
  currentPage: number;                 // 現在のページ番号 (数値)
  totalPages: number;                 // 全ページ数 (数値)
  setCurrentPage: (page: number) => void; // ページを変える「関数」を受け取る
  buttonColor?: string;               // ボタン色 (? は「省略可能」の意味)
  hoverColor?: string;                // ホバー色 (省略可能)
}
```

- `currentPage: number` … 「`currentPage` という名前で、数値 (number) を受け取る」。
- `setCurrentPage: (page: number) => void` … なんと **関数** も props として渡せます。「数値を1つ受け取り、何も返さない (void) 関数」という型。親の `setCurrentPage` を子に渡し、子から呼んでもらうのです (§9で詳説)。
- `buttonColor?: string` … `?` が付くと **省略可能 (オプション)**。渡されなくてもエラーになりません。

用語: interface (インターフェース) TypeScriptで「オブジェクトの形 (どんな名前のプロパティが、どんな型で入っているか)」を定義する設計図。props や APIの戻り値の形を定義するのに多使します。

3.4 propsを「分割代入」で受け取る

定義した props を、関数の引数で受け取ります。 `Pagination.tsx` の実コード。

```
const Pagination: React.FC<PaginationProps> = ({
  currentPage,           // props から currentPage を取り出す
  totalPages,            // totalPages を取り出す
  setCurrentPage,        // 関数 setCurrentPage を取り出す
  buttonColor = "#fff",  // buttonColor。渡されなければ既定値 "#fff" (白)
  hoverColor = "#fff",   // hoverColor。渡されなければ既定値 "#fff"
}) => {
```

- `React.FC<PaginationProps>` … 「`PaginationProps` という形の props を受け取る関数コンポーネント」。 `< >` の中に props の型を指定します。
- `({ currentPage, totalPages, ... })` … 第4章で学んだ **分割代入**。props オブジェクトから必要な値を名前で取り出しています。
- `buttonColor = "#fff"` … **デフォルト値**。親が `buttonColor` を渡さなかったとき、白 (`#fff`) が使われます。`KosuList` は `buttonColor="#0ff"` (水色) を渡すので、こちらが優先されます。

§8と§9で、`KosuList` (親) が `Pagination` (子) に props を渡し、子がそれを使う流れを完全に追跡します。

4. state と useState — 画面が覚える値

4.1 stateとは — 「変化する、覚えておく値」

用語: state (ステート：状態) コンポーネントが内部で **覚えておき、変化したら画面を作り直す** 値のこと。「現在のページ番号」「読み込み中かどうか」「入力された従業員番号」など、ユーザーの操作や通信で変わる値です。

propsとの違いが大切です。

	props	state
誰のもの	親が持ち、子に渡す	自分 (その部品) が持つ

変えられる？	子には変えられない（読み取り専用）	自分で変えられる
変えると	（親が変える）	その部品が再描画される

たとえば：props は「上司から渡された指示書（自分では書き換えられない）」、state は「自分の手元のメモ帳（自分で書き換える。書き換えると画面に反映）」です。

4.2 useState の文法

stateを作るには `useState`（ユーズステート）という関数を使います。

用語: フック (hook) `use` で始まるReactの特別な関数の総称。 `useState` ・ `useEffect` ・ `useRef` ・ `useCallback` など。コンポーネントに「stateを持つ」「副作用を起こす」などの能力を“引っ掛ける (hook)”ためのもの。**必ずコンポーネント関数の先頭（トップレベル）で呼ぶ**のがルールです（if文やループの中ではNG）。

`KosuList.tsx` の実コードで文法を見ます。

▼ **このコードがやること（先に日本語で）**：「ページ番号」を覚えるstateを1つ作ります。初期値は 1。 `currentPage` で今の値を読み、 `setCurrentPage(新しい値)` で書き換えます。

```
const [currentPage, setCurrentPage] = useState<number>(1);
```

分解すると：

- `useState<number>(1)` … 「数値 (number) 型のstateを作る。初期値は **1**」。
- `[currentPage, setCurrentPage]` … `useState` は **2つの値が入った配列** を返し、それを分割代入で受け取ります。
 - `currentPage` … **今の値**（読み取り用）。最初は **1**。
 - `setCurrentPage` … **値を更新する専用の関数**（更新用）。 `setCurrentPage(2)` と呼ぶと、値が2になり、**画面が自動で作り直されます**。

⚠ **stateは直接書き換えてはいけない**。 `currentPage = 2;` のように直接代入してもReactは気づかず、画面は更新されません。**必ず `setCurrentPage(2)` のように更新関数を使います**。

これがReactに「値が変わったよ、画面を作り直して」と伝える唯一の方法です。

4.3 「更新関数を呼ぶと再描画される」のしくみ

これがReactの心臓部です。流れを図にします。

- ① setCurrentPage(2) を呼ぶ
▼
- ② Reactが「currentPage が 2 になった」と記録
▼
- ③ KosuList 関数が もう一度 最初から実行される（再描画）
▼
- ④ 今度は currentPage が 2 の状態でJSXが作られる
▼
- ⑤ 仮想DOMで差分を計算し、変わった所だけ本物の画面に反映

超重要な気づき: コンポーネント関数（`KosuList`）は **一度だけ実行されるのではなく、state が変わるたびに何度も最初から実行されます**。だから「画面 = 今のstateから計算される結果」という関係が常に保たれます。これがReactの宣言的（§ 0.2）の正体です。

4.4 本アプリで使っているstate一覧（KosuList）

`KosuList.tsx` が持つstateを先に俯瞰します（§ 8で1つずつ解説）。

```
const [data, setData] = useState<Kosu[]>([]); // 表に出す工数データ
const [loading, setLoading] = useState<boolean>(true); // 読み込み中か。初期
const [error, setError] = useState<string | null>(null); // エラー文。初期は null
const [searchByMonth, setSearchByMonth] = useState<boolean>(false); // 月で検索するか。初期
const [currentPage, setCurrentPage] = useState<number>(1); // 現在ページ。初期 1
const [totalPages, setTotalPages] = useState<number>(0); // 全ページ数。初期 0
```

`Kosu[]` ・ `boolean` ・ `string | null` ・ `number` のように、stateごとに型を指定しています。

`string | null` は「文字列 **または** null」という意味の **ユニオン型**（第4章）です。

5. useEffect — タイミングで処理を走らせる

5.1 useEffectとは — 「描画の“ついでに”やる処理」

用語: useEffect (ユーズエフェクト) / 副作用 (side effect) 「画面を描く」以外の処理——**サーバーから通信でデータを取る・ページタイトルを変える・タイマーを仕掛ける・イベントを登録する**——を、適切なタイミングで実行するためのフック。これらを「副作用」と呼びます。

なぜ専用のフックが必要なのか。§ 4.3で見たように、コンポーネント関数は再描画のたびに何度も実行されます。その本体に直接 `api.get(...)` (通信) を書くと、**再描画のたびに通信が走り** 無限ループや無駄が発生します。 `useEffect` は「描画とは別のタイミングで、必要なときだけ」副作用を走らせる仕組みです。

5.2 useEffect の文法と「依存配列」

```
useEffect(() => {  
  // ここに副作用の処理 (通信・タイマーなど)  
}, [依存する値1, 依存する値2]); // ← この [] が「依存配列」
```

- 第1引数：実行したい処理 (関数)。
- 第2引数：**依存配列 (dependency array)**。「この配列の中の値が **変わったとき** だけ、上の処理を再実行する」という指定。

用語: 依存配列 (いそんはいれつ) `useEffect` の第2引数 `[ ... ]`。中に入れた値が前回と変わったときだけ、effectが再実行される。

- `[]` (空配列) … **最初の1回だけ** 実行 (マウント時のみ)。
- `[a, b]` … `a` か `b` が変わるたびに実行。
- 第2引数を **書かない** … **毎回の描画後に** 実行 (ほぼ使わない。無限ループの危険)。

たとえ話：依存配列は「**目覚まし時計のセット**」です。 `[currentPage]` は「ページ番号が変わったら鳴って (=処理を実行して)」というアラーム設定。何も変わらなければ鳴りません。

5.3 本アプリの2つの useEffect (予告)

`KosuList.tsx` には `useEffect` が2つあります。§8で完全解説しますが、役割を先に示します。

① 画面に来たらリセットする effect

```
useEffect(() => {
  searchDayRef.current = ""; // 検索日をクリア
  // ...入力欄もクリア、月検索フラグも、ページも1に戻す...
}, [location.pathname]); // URLのパスが変わったとき (=この画面に来たとき)
```

② データを取りに行く effect

```
useEffect(() => {
  fetchData(currentPage, searchDayRef.current, searchByMonth); // サーバーからデータ取得
}, [currentPage, fetchData, searchByMonth]); // ページ・取得関数・月フラグが変わるたびに
```

「ページ番号が変わったら (=ページ送りしたら)、自動で新しいページのデータを取りに行く」
——これが②の効果です。ユーザーがページ送りボタンを押す → `setCurrentPage` でstateが変わる
→ ②のeffectが反応して通信、という連鎖が起きます。

5.4 クリーンアップ (後片付け)

`useEffect` の処理が **関数を return** すると、それは「**後片付け (クリーンアップ) 関数**」になります。次にeffectが再実行される前、または部品が消える (アンマウント) ときに呼ばれます。

`TableContainer.tsx` の実例。

```
useEffect(() => {
  window.addEventListener("resize", updateDimensions); // ウィンドウのリサイズを監視開始
  return () => { // ← 後片付け関数
    window.removeEventListener("resize", updateDimensions); // 監視を解除 (メモリリーク防止)
  };
}, [...]);
```

なぜ後片付けが要るの？ イベント監視やタイマーを仕掛けたまま部品が消えると、存在しない部品に向かって処理が走り続け、メモリの無駄 (メモリリーク) やエラーになります。「仕掛け

たら、必ず外す」。Loading.tsx の clearTimeout、TableContainer.tsx の removeEventListener がこれです (§ 10・§ 11)。

6. useRef — 再描画を起こさない箱

6.1 useRefとは — 「覚えるけど画面は更新しない箱」

用語: useRef (ユーズレフ／reference：参照) 値を保持する「箱」だが、**その中身を書き換えても再描画は起きない**のが特徴。`.current` というプロパティに値が入っている。用途は2つ：(A) 再描画不要な値を覚える、(B) DOM要素 (input・divなど) を直接つかむ。

useStateとの違いが核心です。

	useState	useRef
値を変えると	再描画が起きる	再描画は 起きない
値の置き場所	[値, 更新関数]	<code>ref.current</code>
使いどころ	画面に出る値	画面に出さない値・DOM操作

6.2 用途A：再描画不要な値を覚える

`KosuList.tsx` の検索日の保持。

```
const searchDayRef = useRef<string>(""); // 検索する日付を覚える箱。初期は空文字
// ...
onChange={e => { searchDayRef.current = e.target.value; }} // 入力が変わるたび箱の中身
```

なぜ state ではなく ref? 日付入力欄に文字を打つたびに `setState` すると毎回再描画が走り、重くなります。検索日は「検索ボタンを押した瞬間に使えばよい」ので、**打っている間は再描画不要** → `useRef` で十分。ボタンを押したとき `searchDayRef.current` を読んで通信します。「入力中はこっそり覚えるだけ」の使い分けです。

6.3 用途B：DOM要素を直接つかむ

`KosuList.tsx` では、表の高さ計算のために検索バーと見出しの **実際のDOM要素** をつかんでいます。

```
const searchBarRef = useRef<HTMLDivElement>(null); // <div>（検索バー）をつかむ箱
const headerRef = useRef<HTMLHeadingElement>(null); // <h1>（見出し）をつかむ箱
// ...
<div ref={searchBarRef} className={styles["search-bar"]} > // ref を要素に取り付ける
```

`ref={searchBarRef}` と要素に取り付けると、`searchBarRef.current` でその **本物のDOM要素** に触れます。`TableContainer.tsx` では `searchBarRef.current?.offsetHeight`（その要素の高さ）を読んで、表の最大高さを計算しています（§11）。

用語: マウント／アンマウント マウント = コンポーネントが画面に初めて現れること。**アンマウント** = 画面から取り除かれること。`ref` でDOMをつかめるのはマウント後です。だから初期値は `null`（まだ要素がない）にしておきます。

7. useCallback — 関数を使い回す

7.1 問題：再描画のたびに関数が「作り直される」

§4.3で「コンポーネント関数は再描画のたびに最初から実行される」と学びました。すると、その中で定義した関数（例 `fetchData`）も **毎回新しく作り直されます**。中身は同じでも、JavaScript的には「別物の関数」として扱われます。

これが `useEffect` の依存配列と相性が悪いのです。`fetchData` を依存配列に入れていると、「毎回 `fetchData` が別物になる → effectが毎回再実行 → 無限ループ」の危険があります。

7.2 useCallback — 「中身が同じなら同じ関数を使い回す」

用語: useCallback（ユーザコールバック） 関数を **メモ化（memoization：記憶して使い回す）** するフック。依存配列の値が変わらない限り、**前回と同じ関数オブジェクトを返す**。再描

画のたびに関数が作り直されるのを防ぎ、`useEffect` の不要な再実行を抑える。

`KosuList.tsx` の `fetchData` がこれです。

```
const fetchData = useCallback(async (page, day, mode) => {  
  // ...通信処理...  
}, [navigate]); // navigate が変わらない限り、同じ fetchData を使い回す
```

`[navigate]` が依存配列。`navigate`（画面遷移関数）は基本変わらないので、`fetchData` は **ずっと同じ関数** として保たれます。だから `useEffect` の依存配列 `[currentPage, fetchData, searchByMonth]` に `fetchData` を入れても、無限ループになりません。

たとえ話: `useCallback` は「**同じレシピなら、同じ料理人を使い回す**」こと。客（`useEffect`）は料理人（`fetchData`）が交代したかを毎回チェックしますが、`useCallback` のおかげで料理人が変わらず、客は「同じ人だから注文し直さなくていい」と安心できます。

§8で `fetchData` の中身を1行ずつ完全解説します。

8. 実コード全解説：KosuList.tsx 全215行

いよいよ本章の核心です。本アプリの **工数一覧画面** `frontend/src/KosuPage/KosuList.tsx` を、**1行目から215行目まで、1行も飛ばさず** 読みます。これまで学んだ全部品（コンポーネント・JSX・props・state・`useEffect`・`useRef`・`useCallback`）が、ここに実物として揃っています。

8.1 import群（1～8行目）

▼ **このブロックがやること:** この画面が使う「道具」を、他のファイルから取り込みます。Reactのフック群、通信の道具、画面遷移の道具、表示部品、専用スタイルです。

```
import React, { useState, useEffect, useCallback, useRef } from "react"; // ① Reactと、使
import api from "../api/axios"; // ② 通信の共通
import axios from "axios"; // ③ エラー判定
import { Link, useNavigate, useLocation } from "react-router-dom"; // ④ 画面遷移
import TableContainer from "../Components/TableContainer"; // ⑤ スクロール
import Pagination from "../Components/Pagination"; // ⑥ ページ送り
import Loading from "../Components/Loading"; // ⑦ 読み込み中
import styles from "../styles/KosuPage/KosuList.module.css"; // ⑧ この画面専
```

1行ずつ：

- ① `import React, { useState, useEffect, useCallback, useRef } from "react"` `react` 本体から、`React` と、**名前付き**で4つのフックを取り込みます。`{ }` で囲むのが名前付きインポート（第4章）。この画面はstate・副作用・メモ化・参照の全部を使うので、4つとも取り込んでいます。
- ② `import api from "../api/axios"` 自作の通信設定 `api`（§12で全解説）を取り込みます。`api.get(...)` でサーバーと話します。`../` は「1つ上のフォルダ」。
- ③ `import axios from "axios"` 本家の `axios` 本体。`axios.isAxiosError(err)`（後述）という**エラーがaxios由来か判定する関数**を使うために、別途取り込んでいます。②の `api` とは役割が違います。
- ④ `import { Link, useNavigate, useLocation } from "react-router-dom"` 画面遷移ライブラリ（§13・§14）から3つ。`Link` = リンク部品、`useNavigate` = プログラムから画面移動、`useLocation` = 今どのURLにいるかを知る。
- ⑤～⑦ 自作の表示部品3つ。`TableContainer`（表を画面内に収めてスクロールさせる枠）、`Pagination`（ページ送り）、`Loading`（読み込み中のヘルメット）。
- ⑧ `import styles from "..."` この画面専用のCSS Modules（第3章 §13）。`styles["kosu-list-wrapper"]` のように使います。

8.2 データの型定義 interface Kosu（10～17行目）

▼ **このブロックがやること**：サーバーから受け取る「工数1件分」のデータの形を、TypeScriptの設計図として定義します。これで「このデータには `name` がある」等が型で保証されます。

```
interface Kosu { // 工数1件分のデータの形（設計図）
  id: number; // レコードの一意なID（数値）
  employee_no3: number; // 従業員番号（数値）
  name: string; // 氏名（文字列）
  work_day2: string; // 就業日（"2025-06-01" のような文字列）
  tyoku2: string; // 直（ちよく：勤務シフト区分。"1"～"6" の文字列）
  judgement: boolean; // 整合性の判定（true=OK / false=NG）
}
```

- `interface Kosu { ... }` … 「`Kosu` という形のデータ」を定義。サーバーから来るJSONの1件分に対応します。
- 各行は `プロパティ名: 型;`。 `id` は数値、 `name` は文字列、 `judgement` は真偽値（boolean）。
- これを §8.5 で `useState<Kosu[]>([])` のように使い、「`Kosu` の **配列**」という型にします。

なぜ型を定義するの？ サーバーから来るデータの「形」を約束しておくと、`item.name`（スペルミス）と書いた瞬間にエディタが赤線で教えてくれます。実行する前にバグを発見できる——これがTypeScript（第4章）の恩恵です。

8.3 変換関数 `formatTyoku`（19～29行目）

▼ **このブロックがやること:** サーバーから来る「直（シフト）」の数値コード（1～6）を、人が読める日本語（「1直」「常昼」など）に変換します。コンポーネントの外に置かれた、ただの便利関数です。

```
const formatTyoku = (value: string | number): string => { // 直コードを日本語に変換する関数
  switch (Number(value)) { // value を数値に変換して場合分け
    case 1: return "1直"; // 1 なら「1直」
    case 2: return "2直"; // 2 なら「2直」
    case 3: return "3直"; // 3 なら「3直」
    case 4: return "常昼"; // 4 なら「常昼（じょうちゅう）」
    case 5: return "連1直"; // 5 なら「連1直」
    case 6: return "連2直"; // 6 なら「連2直」
    default: return ""; // どれでもなければ空文字
  }
};
```

- `(value: string | number): string` … 「文字列か数値を受け取り、文字列を返す」関数。サーバーが文字列で送っても数値で送っても対応できるよう、ユニオン型で受けています。
- `switch (Number(value))` … `value` を `Number(...)` で数値化してから `switch` で場合分け（第4章）。`"1"`（文字列）も `1`（数値）も同じ `case 1` に入ります。
- `case 1: return "1直";` … 1なら「1直」を返してその場で関数終了。
- `default: return "";` … 想定外の値なら空文字。画面が壊れないための保険です。

なぜコンポーネントの外に書くの？ この関数はstateやpropsに依存しない「純粋な変換」です。コンポーネントの外（モジュールのトップレベル）に置けば、**再描画のたびに作り直されず** 効率的。「画面に依存しない道具は外に」が定石です。

後述しますが、これは表のセルで `{formatTyoku(item.tyoku2)}` のように呼ばれ、`"1" → "1直"` と表示されます。

8.4 変換関数 `getDayOfWeek`（31～35行目）

▼ **このブロックがやること:** "2025-06-01" のような日付文字列から、曜日（「日」「月」…）を求めます。

```
const getDayOfWeek = (dateStr: string): string => {           // 日付文字列→曜日を返す関数
  const days = ["日", "月", "火", "水", "木", "金", "土"]; // 0~6 に対応する曜日の配列
  const date = new Date(dateStr);                          // 文字列から Date オブジェクト
  return days[date.getDay()] || "";                         // getDay() は曜日番号(0=日)。
};
```

- `const days = [...]` … 曜日名の配列。添字0が「日」、1が「月」……6が「土」。
- `new Date(dateStr)` … 日付文字列を、JavaScriptの **Dateオブジェクト**（日付を扱う道具）に変換。
- `date.getDay()` … その日付の **曜日番号** を返すメソッド。`0` = 日曜、`1` = 月曜……`6` = 土曜。
- `days[date.getDay()]` … 曜日番号で配列を引き、曜日名を得ます。例：`getDay()` が `0` → `days[0]` → `"日"`。
- `|| ""` … もし日付が不正で `undefined` になっても、空文字を返す保険（第4章の `||`）。

用語: メソッド (method) オブジェクトが持つ「動詞 (できること)」。 `date.getDay()` の `getDay()` は、Dateオブジェクトの「曜日を教えて」というメソッドです。 **オブジェクト.メソッド名()** の形で呼びます。

8.5 コンポーネント開始とstate宣言 (37~46行目)

```
const KosuList: React.FC = () => {                                // ① 工数一覧コンポーネント開始
  const [data, setData] = useState<Kosu[]>([]);                  // ② 表データの配列。初期は空配列
  const [loading, setLoading] = useState<boolean>(true);         // ③ 読み込み中フラグ。初期 true
  const [error, setError] = useState<string | null>(null);       // ④ エラー文。初期 null
  const searchDayRef = useRef<string>("");                       // ⑤ 検索日の箱 (再描画なし)。
  const [searchByMonth, setSearchByMonth] = useState<boolean>(false); // ⑥ 月検索か。初期 false
  const [currentPage, setCurrentPage] = useState<number>(1);    // ⑦ 現在ページ。初期 1
  const [totalPages, setTotalPages] = useState<number>(0);      // ⑧ 全ページ数。初期 0
  const location = useLocation();                                // ⑨ 現在のURL情報を取得
  const navigate = useNavigate();                                // ⑩ 画面遷移する関数を取得
```

1行ずつ：

- ① `const KosuList: React.FC = () => {` … コンポーネント本体の開始。 `React.FC` で「Reactの関数コンポーネント」と宣言 (§ 1.2)。
- ② `const [data, setData] = useState<Kosu[]>([])` … 表に出す工数データ。型は `Kosu[]` (§ 8.2の `Kosu` の配列)。初期値は **空配列 `[]` ** (まだデータがない)。
- ③ `const [loading, setLoading] = useState<boolean>(true)` … 「読み込み中か」を覚えるstate。 **初期値 `true` ** (最初は必ず読み込み中から始まる)。データが届いたら `false` にします。
- ④ `const [error, setError] = useState<string | null>(null)` … エラーメッセージ。 `string | null` 型で、初期は `null` (エラーなし)。通信失敗時に文字を入れます。
- ⑤ `const searchDayRef = useRef<string>("")` … 検索する日付を覚える `ref` (§ 6.2)。入力中に再描画したくないのでstateではなくref。
- ⑥ `const [searchByMonth, setSearchByMonth] = useState<boolean>(false)` … 「指定月で検索するか／指定日で検索するか」。初期 `false` (日検索)。
- ⑦ `const [currentPage, setCurrentPage] = useState<number>(1)` … 現在のページ番号。初期 `1`。
- ⑧ `const [totalPages, setTotalPages] = useState<number>(0)` … 全部で何ページあるか。初期 `0`。通信でデータ件数が分かってから計算してセットします。

- ⑨ `const location = useLocation()` … React Routerのフック。今どのURLにいるかの情報（`location.pathname` など）を返す。§ 8.8のリセットeffectで使います。
- ⑩ `const navigate = useNavigate()` … React Routerのフック。`navigate("/login")` のように **プログラムから画面を移動** する関数を返す（§ 14）。401エラー時にログイン画面へ飛ばすのに使います。

8.6 DOM参照用のref（48～49行目）

```
const searchBarRef = useRef<HTMLDivElement>(null); // 検索バーの<div>をつかむ箱。初期
const headerRef = useRef<HTMLHeadingElement>(null); // 見出しの<h1>をつかむ箱。初期 nu
```

- `useRef<HTMLDivElement>(null)` … **DOM要素をつかむ** ためのref（§ 6.3）。型は `HTMLDivElement`（div要素）。
- 初期値が `null`なのは、この時点ではまだ要素が画面に存在しないから（§ 6.3のマウント説明）。
- これらは後で `<div ref={searchBarRef}>`・`<h1 ref={headerRef}>` と要素に取り付けられ（§ 8.12・§ 8.13）、`TableContainer` に渡されて表の高さ計算に使われます（§ 8.14・§ 11）。

8.7 fetchData データ取得関数（51～84行目）

この画面の **通信の心臓部** です。サーバーから工数データを取りに行く関数を、`useCallback`（§ 7）でメモ化しています。

▼ **このブロックがやること**: サーバーの `/api/kosu_list/` に「何ページ目の、（検索があれば）この日付・この検索モードのデータをください」とお願いし、返ってきたデータを表用のstateに入れます。途中で読み込み中フラグを立て、エラーは種類別に処理します。

```

const fetchData = useCallback(async (
  page: number,
  day: string,
  mode: boolean
) => {
  setLoading(true);
  try {
    const response = await api.get("/api/kosu_list/", {
      params: {
        page: page,
        ...(day && {
          day: day,
          mode: mode ? "month" : "day",
          filter: "true",
        })
      }
    });
  }
});

const results = response.data.results || []; // ⑦ 応答からデータ配列を取り出す (なければ[])
const pageSize = response.data.page_size || 20; // ⑧ 1ページあたり件数 (なければ20)
setData(results); // ⑨ 表データ stateを更新 (→再描画)
setTotalPages(Math.ceil(response.data.count / pageSize)); // ⑩ 全ページ数を計算してセット
} catch (err) {
  // ⑪ 通信が失敗したらここへ
  if (axios.isAxiosError(err)) {
    // ⑫ エラーがaxios由来 (通信エラー) か
    if (err.response?.status === 401) navigate("/login"); // ⑬ 401=未ログイン→ログイン画面
    else if (err.response?.status === 403) navigate("/"); // ⑬ 403=権限なし→トップ画面
    else setError(err.response?.data.message); // ⑬ その他はエラー文を表示
  } else {
    // ⑭ axios由来でない予期せぬエラー
    setError("不明なエラーが発生しました。IT担当者に連絡してください。"); // ⑭ 汎用メッセージ
  }
} finally {
  // ⑮ 成功でも失敗でも必ず実行
  setLoading(false); // ⑮ 読み込み中フラグを下ろす (ヘルメット)
}
}, [navigate]); // ⑯ 依存配列: navigate が変わらなければ再実行

```

1行ずつついでに：

- ① `const fetchData = useCallback(async (...) => { useCallback` でメモ化 (§7)。`async` は「非同期関数」(第4章)。`await` で通信の応答を待てます。
- ②④⑤⑥ の `page` ・ `day` ・ `mode` … この関数の引数。「どのページの、どの検索条件で取るか」を呼び出し側が指定します。
- ② `setLoading(true)` … 通信開始の合図。これで `loading` が `true` になり、§8.11の早期リターンで **ヘルメットのローディング画面** が出ます。

- ③ `try {` … 通信は失敗しうるので `try/catch` (第4章) で囲みます。
- ④ `const response = await api.get("/api/kosu_list/", {...})` `api` (§12) でサーバーの `/api/kosu_list/` に **GETリクエスト** (データ取得の依頼)。 `await` で **応答が返るまで待ち**、`response` に受け取ります。

用語: GET (ゲット) リクエスト HTTP通信の種類の一つ。「データをください (取得)」という依頼。 `api.get(...)` がこれ。対して「データを送る・登録する」のは `api.post(...)` (POST。 `Login.tsx` のログインで使用)。

- ⑤ `params: { ... }` … URLの末尾に付ける **クエリパラメータ** (検索条件)。 `?page=2&day=...` のような形でサーバーに渡ります。
- ⑥ `...(day && { ... })` … ここが少し高度。 `day` (検索日) が空でないときだけ、 `{ day, mode, filter }` を **スプレッド (...)** で展開して `params` に追加 します。 `day` が空 (検索なし) なら `day && { ... }` が空になり、何も追加されません。

用語: スプレッド構文 (...) オブジェクトや配列を「中身ごとばらまく」記法 (第4章)。 `...{a:1, b:2}` は `a:1, b:2` を周りに展開します。ここでは「検索日があるときだけ検索条件を差し込む」条件付き展開に使っています。

- ⑥ `mode: mode ? "month" : "day"` … 月検索フラグ (boolean) を、サーバーが分かる文字列 `"month" / "day"` に変換 (三項演算子)。
- ⑦ `const results = response.data.results || []` サーバーの応答 `response.data` の中の `results` (データ配列) を取り出す。 `|| []` で、もし無ければ空配列にして安全に。

用語: response.data axiosは、サーバーの応答全体を `response` に入れ、その本体 (JSONデータ) を `response.data` に入れます。「封筒 (response) の中の手紙 (data)」のイメージ。

- ⑧ `const pageSize = response.data.page_size || 20` … 1ページの件数。サーバーが教えてくれなければ20とみなす。
- ⑨ `setData(results)` … 取得したデータを `data` stateにセット → **再描画** され、表に行が並びます。

- ⑩ `setTotalPages(Math.ceil(response.data.count / pageSize))` 全ページ数を計算してセット。`count` (総件数) ÷ `pageSize` (1ページ件数) を `Math.ceil` (切り上げ)。例: 53件 ÷ 20 = 2.65 → 切り上げて **3ページ**。半端な件数でも最後の1ページを確保するため切り上げます。

用語: Math.ceil (マストット・シール) 「天井方向に切り上げる」関数。

`Math.ceil(2.1)` も `Math.ceil(2.9)` も **3**。`ceil` は ceiling (天井)。

- ⑪ `catch (err) {` … ④の通信が失敗すると、ここに `err` (エラー情報) が飛んできます。
- ⑫ `if (axios.isAxiosError(err))` … `import axios` (③) の関数で「このエラーは通信(axios)由来か?」を判定。サーバーが返した401・403などを見るために使います。
- ⑬ `if (err.response?.status === 401) navigate("/login")` 応答のHTTPステータスが **401** (未認証 = ログインしていない) なら、`navigate` でログイン画面へ強制移動。
 - `err.response?.status` の `?.` は **オプショナルチェーン** (第4章)。「`response`」があればその `status` を見る、なければ `undefined`。応答が来ない通信断でも落ちないための安全装置。

用語: HTTPステータスコード サーバーが「結果の種類」を返す3桁の番号。**200**=成功、**401**=未ログイン、**403**=権限なし、**404**=見つからない、**500**=サーバー内部エラー。本アプリは401でログインへ、403でトップへ誘導します (CLAUDE.mdの「401はloginへ」方針の実装)。

- ⑭ `else if (err.response?.status === 403) navigate("/")` … **403 (権限なし)** ならトップ画面へ。
- ⑮ `else setError(err.response?.data.message)` … その他のエラーは、サーバーが返したメッセージ (`data.message`) を `error` stateにセット → § 8.11で画面に出ます。
- ⑯⑰ `else { setError("不明な...") }` … そもそも通信由来でない予期せぬエラー (プログラムのバグ等) には、汎用メッセージを表示。
- ⑱ `finally {` … 成功 (try) でも失敗 (catch) でも **必ず** 実行されるブロック (第4章)。
- ⑲ `setLoading(false)` … 読み込み中フラグを下ろす。これで `loading` が `false` になり、ヘルメットが消えて本来の画面 (表など) が出ます。

- ②⑩ `}, [navigate])` … `useCallback` の依存配列。 `navigate` は基本変わらないので、`fetchData` は **同じ関数として使い回され** (§ 7)、§ 8.9のeffectで無限ループになりません。

8.8 useEffect ①リセット (86～92行目)

▼ **このブロックがやること:** この画面に「来た」とき (URLが `/kosu-list` になった瞬間) に、検索条件をすべて初期状態に戻します。前回の検索が残っていると混乱するので、まっさらにしめます。

```
useEffect(() => {
  searchDayRef.current = ""; // ① 検索日の箱を空
  const input = document.getElementById("search-day-input") as HTMLInputElement; // ② 入力欄
  if (input) input.value = ""; // ③ 入力欄の表示を空にする
  setSearchByMonth(false); // ④ 月検索フラグを日検索に戻す
  setCurrentPage(1); // ⑤ ページを1に戻す
}, [location.pathname]); // ⑥ URLのパスが変わったとき
```

- ① `searchDayRef.current = ""` … refの中身 (検索日) を空に。
- ② `const input = document.getElementById("search-day-input") as HTMLInputElement` 日付入力欄を、IDで直接つかみます。 `as HTMLInputElement` は「これは入力欄要素だとTypeScriptに教える」型アサーション (第4章)。
- ③ `if (input) input.value = ""` … 入力欄が見つければ、その表示値を空に。 `defaultValue` でできた入力欄の中身は state ではないので、こうしてDOM経由でクリアします。
- ④ `setSearchByMonth(false)` … 月検索フラグを日検索に戻す。
- ⑤ `setCurrentPage(1)` … ページを1ページ目に戻す。
- ⑥ `}, [location.pathname])` … **依存配列が `location.pathname` **。URLのパス (`/kosu-list` など) が変わったときだけ実行。つまり「他の画面からこの画面に来た」タイミングでリセットが走ります。

なぜリセットが必要? SPA (§ 13) では画面を再読み込みせず切り替えるため、前回の検索条件が残ることがあります。「画面に来たら毎回まっさらに」しておく、ユーザーが常に同じ初期状態から使い始められます。

8.9 useEffect ②データ取得（94～96行目）

▼ **このブロックがやること:** ページ番号や検索モードが変わるたびに、`fetchData` を呼んで新しいデータを取りに行きます。最初の表示時にも1回走ります。

```
useEffect(() => {  
  fetchData(currentPage, searchDayRef.current, searchByMonth); // 現在ページ・検索日・検  
}, [currentPage, fetchData, searchByMonth]); // これらが変わるたびに再
```

- `fetchData(currentPage, searchDayRef.current, searchByMonth)` … § 8.7の取得関数を、今のページ・検索日・検索モードで呼びます。
- ****依存配列 `[currentPage, fetchData, searchByMonth]` ****:
 - `currentPage` が変わる → ページ送り → **新しいページを取得**。
 - `searchByMonth` が変わる → 検索モード切替 → **取り直し**。
 - `fetchData` … § 7で `useCallback` 化したので変わらず、ここに入れても安全。
- 初回マウント時にも1回実行され、最初のデータが表示されます。

連鎖の全体像: ユーザーが「次」ボタンを押す → `Pagination` が `setCurrentPage(2)` を呼ぶ → `currentPage` が2に → **この②effectが反応** → `fetchData(2, ...)` で2ページ目を取得 → `setData` で表更新 → 再描画。 **ボタン1つの裏で、これだけの連鎖が自動で起きています。**

8.10 handleSearch ー検索ボタンの処理（98～102行目）

▼ **このブロックがやること:** 「指定月」「指定日」ボタンが押されたときの処理。検索モードを設定し、1ページ目に戻して、すぐにデータを取り直します。

```
const handleSearch = (isMonthSearch: boolean) => { // 引数：月検索なら true  
  setSearchByMonth(isMonthSearch); // ① 検索モードを更新  
  setCurrentPage(1); // ② ページを1に戻す  
  fetchData(1, searchDayRef.current, isMonthSearch); // ③ 1ページ目を即取得  
};
```

- ① `setSearchByMonth(isMonthSearch)` … 「指定月」 ボタンなら `true`、「指定日」 なら `false`。
- ② `setCurrentPage(1)` … 検索条件が変われば1ページ目から見たいのでリセット。
- ③ `fetchData(1, searchDayRef.current, isMonthSearch)` … その場で1ページ目を取得。

なぜ②と③が両方あるの？ ②の `setCurrentPage(1)` は「すでに1ページ目だった場合」effectが反応しないことがあります（値が1→1で変化なし）。そこで③で **明示的に** 取得を呼び、確実に検索を実行しています。「stateを戻しつつ、念のため直接も呼ぶ」二段構えです。

8.11 早期リターン（104～105行目）

▼ **このコードがやること:** 「まだ読み込み中」「エラー発生」のときは、表ではなく専用の画面を先に返して関数を抜けます。これを「早期リターン（early return）」と呼びます。

```
if (loading) return <div><Loading isLoading={loading} /></div>; // ① 読み込み中ならヘルプ
if (error) return <div>Error: {error}</div>; // ② エラーならエラー文を表示
```

- ① `if (loading) return <div><Loading isLoading={loading} /></div>` `loading` が `true` の間は、`Loading` 部品（§ 10）を表示して関数終了。下の表のJSXには進みません。`isLoading={loading}` は **propsの受け渡し**（§ 3）——子の `Loading` に「今読み込み中だよ」を伝えます。
- ② `if (error) return <div>Error: {error}</div>` `error` に文字が入っていれば、エラー文を表示して終了。

用語: 早期リターン（early return） 関数の途中で条件を満たしたら、その場で `return` して以降の処理を行わない書き方。「読み込み中・エラー・正常」の3状態を、上から順にふるい分けます。ここを過ぎたら「読み込み完了かつエラーなし」が保証され、安心して表を描けます。

8.12 JSX —全体の枠とnav（107～119行目）

ここからが「正常時に描く画面」です。

```

return (
  <>                                     {/* ① フラグメント（余計なdivを省く） */}
    <div className={styles["kosu-list-wrapper"]} >           {/* ② 画面全体を囲む箱（専用スタイル） */}
      <h1
        ref={headerRef}                                     {/* ③ この<h1>をheaderRefでつかう */}
        className={styles["h1-collar"]}
      >
        工数履歴                                           {/* ④ 見出しテキスト */}
      </h1>

      <nav className={styles["kosu-nav"]} >                 {/* ⑤ ナビゲーション領域 */}
        <Link to="/kosu-menu">工数MENU</Link>               {/* ⑥ 工数メニューへのリンク */}
      </nav>
    </div>
  </>
)

```

- ① `<>` … フラグメント（§ 2.5）。
- ② `<div className={styles["kosu-list-wrapper"]} >` … 画面全体のラッパー。 `styles["..."]` はCSS Modules（第3章 § 13）。
- ③ `ref={headerRef}` … § 8.6で作った `headerRef` を、この `<h1>` に取り付け。これで `headerRef.current` がこの見出し要素を指し、`TableContainer` が見出しの高さを測れます。
- ④ `工数履歴` … 画面の見出し文字。
- ⑤ `<nav className={styles["kosu-nav"]} >` … メニューの領域（セマンティックHTML、第3章）。
- ⑥ `<Link to="/kosu-menu">工数MENU</Link>` … React Routerの `Link`（§ 14）。クリックすると **ページ全体を再読み込みせず** に工数メニュー画面へ移動します。 `to="..."` が移動先のパス。

8.13 JSX — 検索バー（121～148行目）

▼ **このブロックがやること:** 日付入力欄と「指定月／指定日」ボタンを並べた検索バーです。入力欄に日付を打ち、ボタンを押すと検索が走ります。

```

<div
  ref={searchBarRef}                                {/* ① 検索バーの<div>をrefでつかむ */
  className={styles["search-bar"]}
>
  <label htmlFor="search-day-input"></label>          {/* ② 入力欄のラベル（中身は空） */
  <input
    id="search-day-input"                             {/* ③ 入力欄のID（labelやリセットが */
    type="date"                                       {/* ④ 日付入力（カレンダー付き） */
    defaultValue={searchDayRef.current}              {/* ⑤ 初期値（refの値）。以後はDOM */
    onChange={(e) => { searchDayRef.current = e.target.value; }} /* ⑥ 入力変化→refに保存 */
    placeholder="日付を選択"                         {/* ⑦ 未入力時の薄い案内文 */
  />

  <div className={styles["button-group"]}             {/* ⑧ ボタンをまとめる箱 */
    <button
      onClick={() => handleSearch(true)}              {/* ⑨ 押すと月検索（true） */
      className="light_blue_button"                  {/* ⑩ global.cssの水色ボタン */
    >
      指定月
    </button>
    <button
      onClick={() => handleSearch(false)}             /* ⑪ 押すと日検索（false） */
      className="light_blue_button"
    >
      指定日
    </button>
  </div>
</div>

```

- ① `ref={searchBarRef}` … § 8.6の `searchBarRef` をこの `<div>` に取り付け。表の高さ計算に使われます (§ 11)。
- ② `<label htmlFor="search-day-input"></label>` … 入力欄に結びつくラベル（中身は空ですが、アクセシビリティ上の関連づけ。第3章 § 15）。
- ③④ `<input id="search-day-input" type="date">` … 日付入力欄（第3章 § 2.2）。カレンダーから日付を選べます。
- ⑤ `defaultValue={searchDayRef.current}` … `value` ではなく `defaultValue` を使っている点に注目。`value`（制御）にするとstate管理が必要ですが、ここはrefで「初期値だけ与え、以後はDOM任せ」の **非制御コンポーネント** にしています。

用語: 制御／非制御コンポーネント 入力欄の値を **state で管理** するのが「制御 (controlled)」(`Login.tsx` の従業員番号がこれ: `value={employee_no}`)。DOMに任せるのが「非制御 (uncontrolled)」(ここの日付: `defaultValue` + ref)。打つたびの再描画を避けたいときは非制御が向きます。

- ⑥ `onChange={(e) => { searchDayRef.current = e.target.value; }}` … 入力が変わるたびに、その値 (`e.target.value`) を **refに保存** (再描画なし、§ 6.2)。
- ⑦ `placeholder="日付を選択"` … 未入力時の案内文。
- ⑧ `<div className={styles["button-group"]} >` … 2つのボタンをまとめる箱。
- ⑨ `onClick={() => handleSearch(true)}` … 「指定月」ボタン。押すと § 8.10 の `handleSearch(true)` (月検索) を実行。

⚠️ `onClick={handleSearch}` ではなく `onClick={() => handleSearch(true)}` **。引数を渡したいので、アロー関数で **包んで** います。 `onClick={handleSearch(true)}` と書くと「描画時に即実行」されてしまい誤り。「クリックされたら呼ぶ関数」を渡すのがポイントです。

- ⑩ `onClick={() => handleSearch(false)}` … 「指定日」ボタン。 `handleSearch(false)` (日検索) を実行。
- `className="light_blue_button"` … **文字列の** クラス名 = global.css (第3章 § 11) の水色ボタンを適用。CSS Modulesの `styles[...]` と混在しています。

8.14 JSX — 条件分岐と表 (150～200行目)

▼ **このブロックがやること:** データが0件なら「No data found.」、1件以上なら表を表示します。表は `TableContainer` で囲んでスクロール可能にし、各行を `map` で生成します。

```

{data.length === 0 ? (
  <p>No data found.</p>
) : (
  <TableContainer
    searchBarRef={searchBarRef}
    headerRef={headerRef}
  >
    <table>
      <thead>
        <tr>
          <th className={styles["th-collar"]} >就業日</th>    {/* ④ 表ヘッダ各列 */}
          <th className={styles["th-collar"]} >直</th>
          <th className={styles["th-collar"]} >整合性</th>
          <th className={styles["th-collar"]} >編集</th>
          <th className={styles["th-collar"]} >削除</th>
        </tr>
      </thead>
      <tbody>
        {data.map((item) => (
          <tr key={item.id}>                                {/* ⑤ データ配列を1行ずつ表の行に渡す */}
                                                                {/* ⑥ keyに一意なidを指定 */}
          <td>{item.work_day2} ({getDayOfWeek(item.work_day2)})</td>    {/* ⑦ 就業日 + 曜日 */}
          <td>{formatTyoku(item.tyoku2)}</td>                {/* ⑧ 直コードを日本語に変換して表示 */}
          <td>
            className={
              item.judgement                                {/* ⑨ 整合性で色クラスを切替 */}
                ? styles["status-ok"]                      {/* true→OK (緑など) */}
                : styles["status-ng"]                      {/* false→NG (赤など) */}
            }
          >
            {item.judgement ? "OK" : "NG"}                  {/* ⑩ OK/NGの文字 */}
          </td>
          <td>
            <Link
              to={` /kosu-update/${item.id}`}                {/* ⑪ 編集画面へ。URLにidを埋め込む */}
              className={styles["a-collar"]}
            >
              編集
            </Link>
          </td>
          <td>
            <Link
              to={` /kosu-delete/${item.id}`}                {/* ⑫ 削除画面へ。URLにidを埋め込む */}
              className={styles["a-collar"]}
            >
              削除
            </Link>
          </td>
        </tr>

```

```

    ))}
  </tbody>
</table>

```

- ① `{data.length === 0 ? (<p>No data found.</p>) : (...)}` … 三項演算子 (§ 2.3)。 `data` の長さが0なら「データなし」、そうでなければ表を描く。
- ② ~ `<TableContainer searchBarRef={...} headerRef={...}>` 表を `TableContainer` (§ 11) で囲みます。 **ここがpropsの受け渡し**： `searchBarRef` と `headerRef` を子に渡し、子が「画面の高さ - 見出し - 検索バー」で表の最大高さを計算します。
- ④ `<th className={styles["th-collar"]} >就業日</th>` … 表の見出しセル5つ（就業日・直・整合性・編集・削除）。第3章 § 11.9の白文字+縁取りスタイルが効きます。
- ⑤ `{data.map((item) => (...))}` … **核心の繰り返し** (§ 2.4)。 `data` 配列の各 `item` を `<tr>` に変換。20件あれば20行できます。
- ⑥ `<tr key={item.id}>` … 各行に `key` (§ 2.4の注意)。 `item.id` は一意なので最適。
- ⑦ `<td>{item.work_day2} ({getDayOfWeek(item.work_day2)})</td>` 就業日と曜日。
`getDayOfWeek` (§ 8.4) を呼び、 `2025-06-01 (日)` のように表示。
- ⑧ `<td>{formatTyoku(item.tyoku2)}</td>` … 直コード（"1" など）を `formatTyoku` (§ 8.3) で "1直" に変換して表示。
- ⑨ `className={item.judgement ? styles["status-ok"] : styles["status-ng"]}` 整合性の判定（`judgement`）で **セルの色クラスを切替**。OKなら `status-ok`、NGなら `status-ng`。三項演算子で `className` 自体を出し分けています。
- ⑩ `{item.judgement ? "OK" : "NG"}` … 表示文字も判定で出し分け。
- ⑪ `<Link to={ /kosu-update/${item.id} }>編集</Link>` 編集画面へのリンク。 ``/kosu-update/${item.id}`` は **テンプレートリテラル**（第4章のバッククォート）。 `item.id` が `42` なら `/kosu-update/42` というURLになります。この `:id` 付きのルートが § 13 の `index.tsx` で定義されています。
- ⑫ `<Link to={ /kosu-delete/${item.id} }>削除</Link>` … 同様に削除画面へ。

これが「データ駆動の画面」：表の行数も、各セルの中身も、リンク先のURLも、**すべて data (state) から計算** されています。 `data` が変われば（ページ送り・検索で）、表も自動で作り直されます。 § 0.2の「宣言的」がここに結実しています。

8.15 JSX —Pagination と末尾（202～216行目）

▼ **このコードがやること:** 表の下にページ送り部品を置き、必要なpropsを渡します。最後に部品を閉じ、コンポーネントを公開します。

```
<Pagination
  currentPage={currentPage}      {/* ① 現在ページを渡す */}
  totalPages={totalPages}        {/* ② 全ページ数を渡す */}
  setCurrentPage={setCurrentPage} {/* ③ ページ更新関数を渡す（子から呼んでもらう） */}
  buttonColor="#0ff"             {/* ④ ボタン色＝水色 */}
  hoverColor="#0af"              {/* ⑤ ホバー色＝濃い水色 */}
/>
</TableContainer>
)}
</div>
</>
);
};

export default KosuList;          {/* ⑥ この画面を外部に公開 */}
```

- ① `currentPage={currentPage}` … 親の `currentPage` state (§8.5⑦) を、子の `Pagination` に props として渡す。子は「今何ページ目か」を表示できます。
- ② `totalPages={totalPages}` … 全ページ数を渡す。子は「2 / 5」のように表示し、最後のページで「次」を無効化できます。
- ③ `setCurrentPage={setCurrentPage}` … 関数を props として渡す (§3.3)。子の `Pagination` が「次」ボタンで `setCurrentPage(3)` を呼ぶと、**親の state が変わり**、§8.9のeffectが反応して3ページ目を取得します。これが「子→親へ変化を伝える」唯一の正しい方法です。
- ④⑤ `buttonColor="#0ff" hoverColor="#0af"` … ボタンの色を文字列で渡す。`Pagination` 側で省略可能 (§3.4のデフォルト値) ですが、ここでは水色を指定。`KosuList` の検索ボタン (`light_blue_button`) と色を揃えています。
- ⑥ `export default KosuList` … この画面を公開。`index.tsx` が `import KosuList from "../KosuPage/KosuList"` で取り込みます (§13)。

これで `KosuList.tsx` 全215行を読み切りました。import → 型 → 変換関数 → state/ref → 通信 (fetchData) → 2つのeffect → 検索処理 → 早期リターン → JSX(nav/検索/表/map/Pagination) という、Reactコンポーネントの **典型的なフルセット** を、1行も飛ばさず追いました。他の一覧画面

(人員一覧・班員一覧など) も、ほぼ同じ構造です。この1ファイルが読めれば、本アプリの一覧画面はすべて読めます。

9. 実コード全解説：Pagination.tsx 全71行

§8で `KosuList` (親) が渡した props を、`Pagination` (子) がどう受け取り使うかを、`frontend/src/Components/Pagination.tsx` 全71行で追います。propsの「渡す側」と「受け取る側」がつながる瞬間です。

9.1 import と props型定義 (1～10行目)

```
import React from "react"; // ① React本体
import styles from "../styles/Components/Pagination.module.css"; // ② この部品専用のCSS

interface PaginationProps { // ③ 受け取るpropsの設計図
  currentPage: number; // 現在ページ (数値)
  totalPages: number; // 全ページ数 (数値)
  setCurrentPage: (page: number) => void; // ページ更新関数 (数値を受け、何も返さな
  buttonColor?: string; // ボタン色 (省略可)
  hoverColor?: string; // ホバー色 (省略可)
}
```

- ③ `interface PaginationProps` … §3.3で先に見た設計図。`KosuList` が渡す5つの props (うち色2つは省略可) の形を定義。
- `setCurrentPage: (page: number) => void` … 親の更新関数を受け取る型。これがあるから、子から親のstateを変えられます。

9.2 propsの受け取りとデフォルト値 (12～18行目)

```
const Pagination: React.FC<PaginationProps> = ({ // PaginationProps型のpropsを受け取る
  currentPage, // 取り出し (§8.15①が入る)
  totalPages, // 取り出し (§8.15②が入る)
  setCurrentPage, // 取り出し (§8.15③が入る)
  buttonColor = "#fff", // 渡されなければ白。今回は"#0fff"が渡
  hoverColor = "#fff", // 渡されなければ白。今回は"#0af"が渡
}) => {
```

`KosuList` が § 8.15 で渡した値が、ここに入ります。対応関係：

KosuList（親）が渡す	Pagination（子）が受ける
<code>currentPage={currentPage}</code>	<code>currentPage</code>
<code>totalPages={totalPages}</code>	<code>totalPages</code>
<code>setCurrentPage={setCurrentPage}</code>	<code>setCurrentPage</code>
<code>buttonColor="#0fff"</code>	<code>buttonColor</code> （既定の <code>#fff</code> を上書き）
<code>hoverColor="#0af"</code>	<code>hoverColor</code> （既定の <code>#fff</code> を上書き）

9.3 ページ操作の関数4つ（19～26行目）

▼ このブロックがやること：「最初／前／次／最後」の4つのボタンが押されたときに、親の `setCurrentPage` を適切な値で呼ぶ関数を用意します。範囲外（1未満・最終超え）にはならないよう守ります。

```
const handleFirstPage = () => setCurrentPage(1);           // ① 最初へ：1ページ目
const handleLastPage = () => setCurrentPage(totalPages);    // ② 最後へ：最終ページ
const handleNextPage = () => {                               // ③ 次へ
  if (currentPage < totalPages) setCurrentPage(currentPage + 1); // 最終未満なら+1
};
const handlePreviousPage = () => {                           // ④ 前へ
  if (currentPage > 1) setCurrentPage(currentPage - 1);      // 1より大きければ-1
};
```

- ① `handleFirstPage` … `setCurrentPage(1)`。親のページを1にする → 親で再取得。
- ② `handleLastPage` … `setCurrentPage(totalPages)`。最終ページへ。
- ③ `handleNextPage` … `currentPage < totalPages`（最終ページでない）ときだけ `+1`。最終ページで「次」を押しても進みすぎない安全装置。
- ④ `handlePreviousPage` … `currentPage > 1`（1ページ目でない）ときだけ `-1`。

重要な再確認：これらはすべて、親から受け取った `setCurrentPage` を呼んでいます。子は自分のstateを持たず、親のstateを動かしているだけ。だから親（`KosuList`）の `currentPage`

が変わり、親のeffect（§ 8.9）が反応してデータを取り直す——**props（下り）と関数呼び出し（上り）でデータが循環**しています。

9.4 JSX とCSS変数の注入（28～67行目）

▼ **このブロックがやること:** ページ送りの見た目を描きます。受け取った色をCSS変数として注入し、5つのボタン（最初・前・現在表示・次・最後）を並べます。端のページでは該当ボタンを無効化します。

```

return (
  <div
    className={styles["pagination"]}
    style={{
      "--btn-bg-color": buttonColor, // ① 受け取った色をCSS変数に注入
      "--btn-hover-color": hoverColor //   ホバー色も注入
    } as React.CSSProperties} // ② TypeScriptに「CSSの形」と教える
  >
    <button
      className={styles["prev-button"]}
      disabled={currentPage === 1} // ③ 1ページ目なら「最初」ボタンを無効化
      onClick={handleFirstPage} //   クリックで最初へ
    >
      最初
    </button>
    <button
      className={styles["prev-button"]}
      disabled={currentPage === 1} // ④ 1ページ目なら「前」も無効化
      onClick={handlePreviousPage}
    >
      前
    </button>
    <span>
      {currentPage} / {totalPages} // ⑤ 「2 / 5」のような現在地表示
    </span>
    <button
      className={styles["next-button"]}
      disabled={currentPage === totalPages} // ⑥ 最終ページなら「次」を無効化
      onClick={handleNextPage}
    >
      次
    </button>
    <button
      className={styles["next-button"]}
      disabled={currentPage === totalPages} // ⑦ 最終ページなら「最後」を無効化
      onClick={handleLastPage}
    >
      最後
    </button>
  </div>
);

```

- ① `style={{ "--btn-bg-color": buttonColor, ... }}` 受け取った色（`#0ff`）を、**CSS変数**（第3章 § 11.5）としてインラインstyleで注入。`Pagination.module.css` 側が `var(--btn-bg-color)` でこの色を使い、ボタンを着色します。**propsで渡した色が、CSSにまで伝わる** 仕組み。

- `style={{ ... }}` の二重中括弧は「JSXの `{ }` (JS埋め込み) の中に、オブジェクト `{ }` を書く」ため (第3章 § 4.3)。
- ② `as React.CSSProperties` … TypeScriptに「これはCSSスタイルのオブジェクト」と教える型アサーション。CSS変数 (`--` 始まり) を許可させるためのおまじない。
- ③ `disabled={currentPage === 1}` `currentPage` が1なら `disabled` (操作不可、第3章 § 1.3)。1ページ目で「最初」「前」を押せないようにします。`disabled` が `true` のボタンは灰色になり押せません。
- ⑤ `<span>{currentPage} / {totalPages}</span>` … 現在地表示。 `2 / 5` のように出ます。 `{ }` でstate値を埋め込み。
- ⑥⑦ `disabled={currentPage === totalPages}` … 最終ページなら「次」「最後」を無効化。
- 各 `onClick` … § 9.3で作った4つの関数を割り当て。クリック → その関数 → `setCurrentPage` → 親のstate変化 → 親のデータ再取得。

9.5 公開 (69～71行目)

```
};

export default Pagination; // この部品を公開 (KosuList等が import する)
```

`Pagination.tsx` 全71行を読み切りました。この部品は **自分のstateを一切持たず**、すべて props (現在ページ・全ページ数・更新関数・色) で動く「純粋な部品」です。だからこそ、工数一覧でも 人員一覧でも 班員一覧でも、`<Pagination .../>` と置くだけで再利用できます。「**propsだけで動く部品 = どこでも使える部品**」の好例です。

10. 実コード全解説：Loading.tsx 全42行

`frontend/src/Components/Loading.tsx` を全解説します。`KosuList` の早期リターン (§ 8.11) で表示される「ヘルメットが揺れる読み込み中画面」です。`useEffect` のクリーンアップ (後片付け) の実例でもあります。

10.1 import と props (1～7行目)

```
import React, { useEffect, useState } from "react"; // React・useEffect・useState
import helmetImage from "../img/helmet.png"; // ① ヘルメット画像を取り込む (ビルド
import styles from "../styles/Components/Loading.module.css"; // 専用CSS

interface LoadingProps {
  isLoading: boolean; // ② 親から「読み込み中か」を受け取る
}
```

- ① `import helmetImage from "../img/helmet.png"` … 画像も `import` で取り込めます (ビルドツールが処理)。`<img src={helmetImage}>` で表示。
- ② `isLoading: boolean` … 親 (`KosulList`) から渡される唯一のprops。 `<Loading isLoading={loading} />` (§ 8.11) で渡された値が入ります。

10.2 フェードアウト用のstate (9～10行目)

```
const Loading: React.FC<LoadingProps> = ({ isLoading }) => { // isLoadingを受け取る
  const [shouldRender, setShouldRender] = useState(true); // 表示すべきか。初期 true
```

- `shouldRender` … 「まだ画面に出しておくか」のstate。読み込みが終わってもすぐ消さず、1秒かけてフェードアウトさせてから消すために使います。

なぜ即消さないの? パッと消えると目がチカチカします。 `isLoading` が false になっても、1秒間フェードアウト演出を見せてから本当に消す——そのための「猶予」を `shouldRender` で管理しています。

10.3 useEffect とクリーンアップ (12～20行目)

▼ **このブロックがやること:** 読み込みが終わったら、1秒後に表示を完全停止します。タイマーを仕掛け、部品が消えるときはタイマーも片付けます。

```

useEffect(() => {
  if (!isLoading) {                                     // ① 読み込みが終わったら
    const timeout = setTimeout(() => {                  // ② 1秒後に実行するタイマーを仕掛ける
      setShouldRender(false);                          // ③ 表示を完全停止
    }, 1000);                                           // 1000ミリ秒=1秒 (CSSトランジションと同)
    return () => clearTimeout(timeout);                 // ④ 後片付け：タイマーを解除 (アンマウント時)
  }
}, [isLoading]);                                       // ⑤ isLoadingが変わるたびに実行

```

- ① `if (!isLoading)` … `isLoading` が `false` (読み込み終了) になったときだけ処理。
- ② `const timeout = setTimeout(() => {...}, 1000)` `setTimeout` は「指定ミリ秒後に1回だけ実行」する標準関数。1000ミリ秒 (1秒) 後に③を実行する予約をし、その予約番号を `timeout` に保存。
- ③ `setShouldRender(false)` … 1秒後、表示を停止 (§ 10.4で `null` を返すように)。
- ④ `return () => clearTimeout(timeout)` … **クリーンアップ関数** (§ 5.4)。`clearTimeout` で仕掛けたタイマーを解除。effectが再実行される前や、部品が消えるときに呼ばれます。

なぜタイマーを片付ける？ タイマー予約が残ったまま部品が消えると、存在しない部品に対して `setShouldRender` が呼ばれ警告・エラーになります。「仕掛けたら外す」の鉄則 (§ 5.4)。

- ⑤ `}, [isLoading])` … 依存配列は `isLoading`。読み込み状態が変わるたびにこのeffectを評価します。

10.4 早期リターンとJSX (22~40行目)

```
if (!shouldRender) return null; // ① 表示停止なら何も描かない (null)

return (
  <div
    id="loading"
    className={` ${styles.loading} ${!isLoading ? styles.fadeOut : ""}`} // ② 読み終了時に
  >
    <div className={styles.item}>
      <img
        id="loadingImg"
        src={helmetImage} // ③ ヘルメット画像
        alt="Loading..." // 代替文字
        className={styles.sway} // 左右に揺れるアニメ
      />
    </div>
    <p className={styles.loadingText}>Loading...</p> // ④ 「Loading...」の文字
  </div>
);
```

- ① `if (!shouldRender) return null` ... `null` を返すと **何も描画しません** (§ 10.3③で1秒後にこうなり、画面から消える)。早期リターン (§ 8.11) の応用。
- ② `className={` ${styles.loading} ${!isLoading ? styles.fadeOut : ""}`}` テンプレートリテラルで **2つのクラスを連結**。常に `styles.loading` を当て、`isLoading` が `false` のとき (読み終了) だけ `styles.fadeOut` (透明になる演出) を追加。三項演算子で条件付きのクラスを足しています。
 - `styles.loading` の書き方は `styles["loading"]` と同じ (ドット記法)。ハイフンを含まないクラス名はこちらでも書けます。
- ③ `<img src={helmetImage} ... className={styles.sway}>` ... § 10.1で取り込んだヘルメット画像を表示。`sway` (揺れ) クラスで左右に揺れます。
- ④ `<p ...>Loading...</p>` ... 「Loading...」の文字。

Loading.tsx 全42行を読み切りました。ポイントは **props (`isLoading`) で親と連動** し、
** `useEffect` + `setTimeout` + クリーンアップ** で「即消さずフェードアウトしてから消す」上品な振る舞いを実現している点です。

11. 実コード全解説：TableContainer.tsx 全57行

`frontend/src/Components/TableContainer.tsx` を全解説します。`KosuList` が表を囲んだ部品 (§ 8.14) で、「画面の高さに合わせて表の枠の高さを自動調整し、はみ出したらスクロール」を実現します。`children` (子要素を受け取る props) と、リサイズ対応のクリーンアップの実例です。

11.1 import と props型 (1~9行目)

```
import React, { useState, useEffect, useRef, ReactNode } from "react"; // ReactNodeも取り扱う
import styles from "../styles/Components/TableContainer.module.css";

interface TableContainerProps {
  children: ReactNode; // ① 中に入れる子要素 (表など)
  searchBarRef: React.RefObject<HTMLInputElement | null>; // ② 検索バーのref
  headerRef: React.RefObject<HTMLInputElement | null>; // ③ 見出しのref
  heightExpansion?: boolean; // ④ 高さ拡張モード (省略可)
}
```

- ① `children: ReactNode` … **特別なprops `children` **。 `<TableContainer>`ここに書いた中身 `</TableContainer>` の「中身」が、この `children` に入ります。`ReactNode` は「JSXとして描けるものなら何でも (要素・文字・配列)」という型。

用語: children (チルドレン) 開始タグと終了タグの **間に書いた中身** を受け取る特別な props。`KosuList` では `<TableContainer>` の中に `<table>` と `<Pagination>` を入れた (§ 8.14) ので、それらが `children` として渡ります。「箱に何でも入れられる」汎用の入れ物を作るための仕組み。

- ②③ `searchBarRef` ・ `headerRef` … 親から渡される ref (§ 8.14③)。検索バーと見出しの **高さ** を測り、表の最大高さ計算に使います。`React.RefObject<HTMLInputElement | null>` は ref 型。
- ④ `heightExpansion?: boolean` … 省略可能なオプション。`true` なら別の計算式を使う (一部画面用)。`KosuList` は渡していないので既定の `false`。

11.2 受け取りと内部state/ref (11~18行目)

```
const TableContainer: React.FC<TableContainerProps> = ({
  children,                // 中身を受け取る
  searchBarRef,            // 検索バーのrefを受け取る
  headerRef,               // 見出しのrefを受け取る
  heightExpansion = false, // 既定 false
}) => {
  const [maxHeight, setMaxHeight] = useState<number>(window.innerHeight); // ① 表枠の最大高さをstateで管理。初期値は window.innerHeight (ブラウザ表示領域の高さ、ピクセル)。
  const containerRef = useRef<HTMLDivElement>(null);                       // ② 自分の枠のref
```

- ① `const [maxHeight, setMaxHeight] = useState<number>(window.innerHeight)` 表枠の最大高さをstateで管理。初期値は `window.innerHeight` (ブラウザ表示領域の高さ、ピクセル)。
- ② `const containerRef = useRef<HTMLDivElement>(null)` ... 自分の枠 `<div>` をつかむref (§ 11.4で取り付け)。

11.3 useEffect — 高さ計算とリサイズ対応 (20~40行目)

▼ **このブロックがやること:** 「画面の高さ - 見出しの高さ - 検索バーの高さ - 余白」を計算して表枠の最大高さに設定します。さらに、ウィンドウのサイズが変わるたびに再計算するよう監視を仕掛け、部品が消えるとき監視を外します。

```

useEffect(() => {
  const updateDimensions = () => { // ① 高さを計算してセット
    const searchBarHeight = searchBarRef.current?.offsetHeight || 0; // ② 検索バーの高さ
    const headerHeight = headerRef.current?.offsetHeight || 0; // ③ 見出しの高さ (が)

    if (heightExpansion) { // ④ 拡張モードなら
      setMaxHeight(window.innerHeight - 100); // 画面高 - 100px
    } else { // ⑤ 通常モードなら
      setMaxHeight(
        window.innerHeight - searchBarHeight - headerHeight - 40 // 画面高 - 検索バー - 見出し
      );
    }
  };

  updateDimensions(); // ⑥ まず1回計算 (初期表示用)

  window.addEventListener("resize", updateDimensions); // ⑦ リサイズのたびに再計算
  return () => { // ⑧ 後片付け
    window.removeEventListener("resize", updateDimensions); // リサイズ監視を解除
  };
}, [searchBarRef, headerRef, heightExpansion]); // ⑨ これらが変わったら再設定

```

- ① `const updateDimensions = () => {...}` … 高さを計算する内部関数。
- ② `searchBarRef.current?.offsetHeight || 0` 親から渡された検索バーrefの **実際の高さ** (`offsetHeight` : その要素の表示高さpx) を取得。 `?.` で「refがまだ空 (null) なら安全に飛ばす」、`|| 0` で「無ければ0」。
- ③ 見出しの高さも同様に取得。
- ④⑤ 表枠の最大高さを計算：
 - 拡張モード (`heightExpansion=true`) なら `画面高 - 100`。
 - 通常モードなら `画面高 - 検索バー高 - 見出し高 - 40`。画面から見出しと検索バーの分を引いた残りを表枠に割り当て、ちょうど画面に収まるようにします。
- ⑥ `updateDimensions()` … 初回表示時に1回計算。
- ⑦ `window.addEventListener("resize", updateDimensions)` ブラウザ窓のサイズ変更 (`resize` イベント) を監視し、変わるたびに `updateDimensions` を呼ぶよう登録。窓を縮めても表枠が追従します。
- ⑧ `return () => { window.removeEventListener(...) }` クリーンアップ (§5.4)。部品が消えるとき監視を解除。これを忘れると、画面を切り替えた後もリスナーが残り続け、メモリリークの原因に。
- ⑨ 依存配列 `[searchBarRef, headerRef, heightExpansion]` … これらが変わったら、effectを張り直します。

11.4 JSX (42～57行目)

```
return (  
  <div  
    ref={containerRef}                // ① 自分の枠をrefでつかむ  
    className={styles["table-wrapper"]}   
    style={{  
      maxHeight: `${maxHeight}px`,    // ② 計算した最大高さを適用  
      overflowY: "auto",              // ③ 縦にはみ出たらスクロールバー  
      overflowX: "auto",              // ④ 横も同様  
    }}  
  >  
    {children}                        // ⑤ 親から渡された中身（表とPagination）を描  
  </div>  
);
```

- ① `ref={containerRef}` … 自分の枠divをrefに紐づけ。
- ② `maxHeight: `${maxHeight}px`` … § 11.3で計算した高さを適用。テンプレートリテラルで `"450px"` のような文字列に。
- ③④ `overflowY: "auto", overflowX: "auto"` … 中身が枠を超えたらスクロールバーを出す（第3章のoverflow）。見出し・検索バーは固定で、表だけがスクロールする使い勝手を実現。
- ⑤ `{children}` … ここが肝。 `KosuList` が `<TableContainer>...</TableContainer>` の中に書いた `<table>` と `<Pagination>` (§ 8.14) が、この位置に描かれます。 `TableContainer` 自身は「中身が何か」を知らず、ただ高さ調整付きの枠を提供するだけ——だからどんな表にも使い回せます。

`TableContainer.tsx` 全57行を読み切りました。 `children` で「中身を問わない汎用の枠」を作り、 `useEffect` + `resize` 監視+クリーンアップで「画面サイズに追従するスクロール枠」を実現しています。 `Pagination` と同じく 再利用可能な部品 の好例です。

12. 実コード全解説：axios.ts 全8行

`frontend/src/api/axios.ts` を全解説します。たった8行ですが、本アプリの全画面がサーバーと話すときの共通設定 が詰まった重要ファイルです。 `KosuList` の `api.get(...)` も `Login` の `api.post(...)` も、この設定を使っています。

用語: axios (アクシオス) ブラウザからサーバーへHTTP通信 (データの取得・送信) を行うための定番ライブラリ。 `api.get(URL)` ・ `api.post(URL, データ)` のように使う。

▼ **このコードがやること:** サーバーの基本URL・Cookie送信・CSRF対策をまとめて設定した「自分専用のaxios (`api`)」を作り、全画面で共有できるよう公開します。

```
import axios from "axios"; // ① axios本体を取り込む
const api = axios.create({ // ② 設定済みの専用インスタンスを作る
  baseURL: process.env.REACT_APP_API_BASE_URL, // ③ 全リクエストの基本URL (環境変数)
  withCredentials: true, // ④ Cookie (ログインセッション) を送る
  xsrfCookieName: "csrftoken", // ⑤ CSRFトークンが入ったCookie名
  xsrfHeaderName: "X-CSRFToken", // ⑥ CSRFトークンを送るヘッダ名
});
export default api; // ⑦ この設定済みapiを全画面に公開
```

1行ずつ：

- ① `import axios from "axios"` … axios本体を取り込み。
- ② `const api = axios.create({ ... })` `axios.create` で「共通設定を持った専用のaxios」を作ります。これを `api` と名付け、各画面は `api.get` ・ `api.post` を使う。設定を1か所にまとめられるのが利点。

用語: インスタンス (instance) 「設定を固めた、その場用のコピー」のこと。

`axios.create` は、毎回URLやCookie設定を書かずに済むよう、共通設定済みのaxiosを1つ作ります。

- ③ `baseURL: process.env.REACT_APP_API_BASE_URL` 全リクエストの **基本URL**。
`api.get("/api/kosu_list/")` と書くと、実際は `baseURL + "/api/kosu_list/"` に送られます。`process.env.REACT_APP_...` は **環境変数** (CLAUDE.md参照)。開発時と本番でサーバーURLを切り替えられます。

用語: 環境変数 (かんきょうへんすう) プログラムの外側 (`.env` ファイルなど) で設定する値。本番サーバーと開発サーバーでURLが違っても、**コードを変えずに** 接続先を切り替

えられます。Reactでは `REACT_APP_` で始まる名前だけが使えます。

- ④ `withCredentials: true` 通信時に **Cookieと一緒に送る** 設定。本アプリはセッション認証（ログイン状態をCookieで保持）なので、これが `true` でないと「ログインしているのに401」になります。 **ログインを維持する命綱**。

用語: Cookie（クッキー）／セッション ログイン後にサーバーが発行する「あなたは誰々さん」という合言葉を、ブラウザが保存しておく小さなデータがCookie。毎回の通信でこれを送ることで、サーバーは「ログイン済みの誰か」を識別します。

- ⑤⑥ `xsrftoken` / `xsrftoken` **CSRF対策**（後述）の設定。 `csrfToken` という名前のCookieに入っている合言葉を読み取り、 `X-CSRFToken` というヘッダに付けて送る、という指定。Django（サーバー側）と名前を合わせています。

用語: CSRF（シーサーフ：Cross-Site Request Forgery） 「なりすまし送信」を防ぐ仕組み。悪意あるサイトが、あなたのログイン状態を悪用して勝手にデータを送る攻撃を防ぎます。サーバーが発行した「合言葉（CSRFトークン）」を、送信時に必ず添える約束にすることで、正規の画面からの送信だけを受け付けます。POST（登録・変更）で特に重要。

- ⑦ `export default api` … この設定済み `api` を公開。 `KosuList` (§8.1②) も `Login` も `import api from "../api/axios"` で取り込み、共通設定で通信します。

`axios.ts` **全8行を読み切りました**。わずか8行に「接続先（baseURL）」「ログイン維持（withCredentials+Cookie）」「なりすまし防止（CSRF）」という、通信のセキュリティと利便性の要が凝縮されています。 **通信がうまくいかないときは、まずこのファイルと環境変数を疑う** のが定石です。

13. ルーティング：index.tsx 完全解説とSPA

`frontend/src/index.tsx` は、アプリの **入口** であり、 **どのURLでどの画面を出すか（ルーティング）** を決める司令塔です。 `KosuList` の `<Link to="/kosu-update/42">` が、なぜ編集画面を開けるのか——その答えがここにあります。

13.1 SPA（シングルページアプリケーション）とは

用語: SPA（エスピーエー：Single Page Application） 「1枚のHTMLページ」だけを最初に読み込み、以降はページ全体を再読み込みせず、**JavaScriptで中身だけを差し替えて** 画面遷移するアプリの作り方。本アプリはSPA。ページ移動が一瞬で、ちらつかないのが特徴。

従来のWebは、リンクをクリックするたびにサーバーから **新しいHTMLを丸ごと取得** し、画面が真っ白→再描画されていました（紙芝居を1枚ずつ差し替えるイメージ）。SPAは、最初に「絵を描く道具一式（JavaScript）」を読み込み、以降は **同じ画面の上で絵だけ描き替える**（ホワイトボードに描いては消すイメージ）。これを実現するのが **React Router** です。

13.2 アプリの起動部分（172～180行目）

▼ **このコードがやること:** HTMLの中の `id="root"` の場所に、Reactアプリ全体を描き込みます。 **Router** で囲むことで、アプリ全体でルーティング（URLに応じた画面切替）が使えるようになります。

```
const root = ReactDOM.createRoot(document.getElementById('root') as HTMLElement); // ① 描

root.render(                                     // ② そこにアプリを描画
  <React.StrictMode>                             // ③ 開発時の問題検出モード
    <Router>                                       // ④ ルーティング機能でアプリ全体を包む
      <App />                                     // ⑤ 全画面を束ねるAppコンポーネント
    </Router>
  </React.StrictMode>
);
```

- ① `const root = ReactDOM.createRoot(document.getElementById('root') as HTMLElement)`
`public/index.html` にある `<div id="root"></div>` を取得し、「ここにReactを描く」と宣言。`as HTMLElement` は型アサーション（要素が必ずある前提）。
- ② `root.render(...)` … その土台に、中身を描画。
- ③ `<React.StrictMode>` … 開発中だけ、潜在的な問題（古い書き方など）を警告してくれる開発支援モード。本番では無効。

用語: StrictMode (厳格モード) 開発中、effectをわざと2回実行するなどして「クリーンアップ忘れ」等のバグをあぶり出すモード。「effectが2回走る？」と驚いたら、これが原因(本番では1回)。

- ④ `<Router>` ... `BrowserRouter` の別名 (3行目で `as Router`)。これで囲んだ中で、URLに応じた画面切替 (`Routes / Route`) や `Link` ・ `useNavigate` が使えます。
- ⑤ `<App />` ... 全ルートを定義するコンポーネント (§13.4)。

13.3 import 群とタイトル設定 (1~115行目)

冒頭は全画面コンポーネントのimport (5~53行目) と、URLに応じた **ブラウザのタブのタイトル** 設定です。要点を抜粋します。

```
import { BrowserRouter as Router, Routes, Route, useLocation } from "react-router-dom";  
import './styles/global.css';  
import Login from './MainPage/Login';           // ③ 以下、全画面コンポーネントを取り込む  
import KosuList from './KosuPage/KosuList';      //      (工数一覧。§8で解説した画面)  
// ... 他多数 ...
```

- ① `import { BrowserRouter as Router, Routes, Route, useLocation } from "react-router-dom"` ルーティングの主役4つ。`BrowserRouter` を `Router` という別名で取り込み (`as`)、`Routes` (ルート一覧の枠)、`Route` (1つの対応)、`useLocation` (現在URL取得)。
- ② `import './styles/global.css'` ... 全画面共通CSS (第3章) を読み込み。`from` がないのは「副作用だけのimport」(読み込んで適用するだけ)。
- ③~ 50以上の画面コンポーネントを取り込み。1画面1ファイルの構成です。

タイトル設定部分 (57~115行目) の要点：

```

const App: React.FC = () => {
  const location = useLocation(); // 現在のURL情報

  React.useEffect(() => {
    const routeTitles: { [key: string]: string } = { // パス→タイトルの対応表
      "/login": "ログイン - 業務工数システム",
      "/kosu-list": "工数一覧 - 業務工数システム",
      // ... 全画面分 ...
    };
    document.title = routeTitles[location.pathname] || "業務工数システム"; // タブのタイ
  }, [location]); // URLが変わるたびに更新

```

- `const location = useLocation()` … 現在のURLを取得 (§ 8.5⑨と同じフック)。
- `routeTitles` … 「URLパス → タブに出すタイトル」の対応表 (オブジェクト)。`{ [key: string]: string }` は「文字列キー・文字列値のオブジェクト」型。
- `document.title = routeTitles[location.pathname] || "業務工数システム"` 現在のパスに対応するタイトルをタブに設定。対応がなければ既定名。`useEffect` の依存配列 `[location]` で、画面が変わるたびにタブ名も自動で変わります。

13.4 Routes と Route (117~169行目)

▼ このコードがやること: 「このURLなら、この画面コンポーネントを表示する」という対応を、URLごとに1行ずつ定義します。URLが変わると、合致した `Route` の画面だけが描かれます。

```

return (
  <Routes> // /* ① ルート一覧の枠 */
    <Route path="/login" element={<Login />} /> // /* ② /login ならLogin画面
    <Route path="/" element={<MainMenu />} /> // /* ③ / ならメインメニュー
    <Route path="/kosu-list" element={<KosuList />} /> // /* ④ /kosu-list ならKosuLi
    <Route path="/kosu-update/:id" element={<KosuEdit />} /> // /* ⑤ :id 付き=可変URL */
    <Route path="/kosu-delete/:id" element={<KosuDelete />} /> // /* ⑥ 削除画面も:id付き */
    /* ... 他50以上のルート ... */
    <Route path="/member-update/:employee_no" element={<MemberEdit />} /> // /* ⑦ 別名の可変
  </Routes>
);

```

- ① `<Routes>` … 複数の `Route` をまとめる枠。中から **今のURLに最初に合致する1つ** を選んで描画します。
- ② `<Route path="/login" element={<Login />} />` 「URLが `/login` なら `<Login />` を表示」。 `path` が条件、 `element` が表示する画面。
- ④ `<Route path="/kosu-list" element={<KosuList />} />` 「 `/kosu-list` なら § 8 で解説した `KosuList` を表示」。 `KosuList` の `<nav>` にある `<Link to="/kosu-menu">` を辿って工数メニューへ、そこからここに来ます。
- ⑤ `<Route path="/kosu-update/:id" element={<KosuEdit />} />` **ここが重要**。 `:id` は **URLパラメータ** (可変部分)。`/kosu-update/42` でも `/kosu-update/99` でも、この1つのルートが受け止め、 `42` や `99` を「id」という名前で取り出せます。`KosuList` の `<Link to={ /kosu-update/{item.id} }>` (§ 8.14⑪) が、ここに繋がります。

用語: URLパラメータ (`:id`) URLの一部を「変数」にする書き方。 `:id` は「ここに何か入る」の意味。編集画面 (`KosuEdit`) では `useParams()` というフックで `const { id } = useParams()` のように取り出し、「どのレコードを編集するか」を判断します。**1つのルート定義で、全レコードの編集画面に対応** できる賢い仕組み。

- ⑦ `:employee_no` … 可変部分の名前は自由。人員編集は `:employee_no` という名前です。

13.5 ルーティングの全体像

`KosuList` の「編集」リンクをクリックしてから編集画面が出るまでの流れ：

- ① ユーザーが `<Link to="/kosu-update/42">` をクリック (§8.14)
▼
- ② React RouterがURLを `/kosu-update/42` に変更 (ページ再読み込みなし=SPA)
▼
- ③ `<Routes>` が、合致するRoute `<Route path="/kosu-update/:id">` を発見
▼
- ④ そのelement `<KosuEdit />` を描画。`:id` の部分 (42) は`useParams`で取得可能
▼
- ⑤ `KosuEdit`が `id=42` のデータを`api.get`で取得し、編集フォームを表示

ページ全体は一切再読み込みされず、中身だけが `KosuList` から `KosuEdit` に差し替わります。これがSPAの軽快さです。

14. Link と useNavigate — 画面の移動

本アプリの画面移動には2つの方法があります。両方ともReact Routerの道具で、§ 13の `<Router>` の中だからこそ使えます。

14.1 `<Link>` — クリックで移動するリンク

用語: Link (リンク) React Routerのリンク部品。HTMLの `<a>` に似ているが、**ページを再読み込みせずSPA的に移動** する点が違う。 `to="移動先パス"` を指定。

`KosuList` の実例：

```
<Link to="/kosu-menu">工数MENU</Link>           { /* メニューへ (§8.12⑥) */ }
<Link to={` /kosu-update/${item.id}`}>編集</Link> { /* 編集へ。idを埋め込む (§8.14⑩) */ }
```

- `to="/kosu-menu"` … 固定の移動先。
- `to={` /kosu-update/${item.id}`}` … テンプレートリテラルで **動的なURL**。各行のidに応じて行き先が変わります。

なぜ `<a href>` でなく `<Link>` ? 素の `<a href="/kosu-menu">` だと、クリックでブラウザがページを **丸ごと再読み込み** (SPAの利点が消える)。`<Link>` は再読み込みせず、Reactが中身だけ差し替えます。****アプリ内の移動は必ず `<Link>` ****、外部サイトへだけ `<a href>`、と覚えましょう。

14.2 `useNavigate` — プログラムから移動

用語: useNavigate (ユーズナビゲート) 「クリック」ではなく **コードの判断で** 画面を移動させるフック。`const navigate = useNavigate()` で関数を得て、`navigate("/login")` のように呼ぶ。

`KosuList` の `fetchData` (§ 8.7) と `Login` の実例：

```
const navigate = useNavigate(); // 移動関数を取得 (§8.5⑩)
// ...
if (err.response?.status === 401) navigate("/login"); // 未ログインなら自動でログイン画面
// ...
if (data.status === "success") navigate("/"); // ログイン成功ならトップへ (Login.)
```

- `navigate("/login")` … リンクのクリックを待たず、**プログラムの判断で** ログイン画面へ飛ばす。401エラー検知時の自動リダイレクトに使います (CLAUDE.mdの「401はloginへ」方針)。
- `navigate("/")` … ログイン成功時、トップ画面へ自動移動。

使い分け: ユーザーが **押して** 移動するなら `<Link>`、**処理結果に応じて自動で** 移動するなら `useNavigate`。本アプリは「メニューリンク = `<Link>`」「ログイン成功・認証切れ = `useNavigate`」と使い分けています。

15. 描画サイクルの全体像（まとめ図）

ここまでの全部品を、`KosuList` でページ送りした瞬間の流れとして1枚にまとめます。

【初回表示】

1. `/kosu-list` にアクセス → `index.tsx` の `<Route>` が `KosuList` を描画 (§13)
2. `KosuList` 実行: state初期化 (`loading=true` 等) (§8.5)
3. `loading=true` なので早期リターンで `<Loading>` 表示 (§8.11・§10)
4. `useEffect`② (§8.9) が発火 → `fetchData(1, "", false)` (§8.7)
5. `api.get` で `/api/kosu_list/` にGET (§12のapi設定でCookie・CSRF付き)
6. 応答到着 → `setData(...)`・`setTotalPages(...)`・`setLoading(false)`
7. state変化で再描画 → `loading=false` → 表 + `Pagination` が表示 (§8.14)

【「次」ボタンを押すと】

8. `Pagination` の「次」→ `handleNextPage` → `setCurrentPage(2)` (§9.3)
※子が、親から渡された `setCurrentPage` を呼ぶ (§3.2の「上りの伝達」)
9. 親の `currentPage` が 2 に → `useEffect`② (依存に `currentPage`) が発火 (§8.9)
10. `fetchData(2, ...)` で2ページ目をGET → `setData` → 再描画 → 表更新

この循環——「**state → 画面 (下り: props)**」と「**操作 → state更新 (上り: 渡された関数)**」——がReactの本質です。`KosuList` と `Pagination` が、まさにこの循環で動いていることを、あな

たはもう1行ずつ確認しました。

16. トラブルシューティング

症状	原因	対処
画面が真っ白	JSの実行エラー（型ミス・undefined参照など）	F12のConsoleで赤いエラーを読む（第3章 § 14.3）。 <code>?.</code> で安全参照を
<code>Too many re-renders</code> 無限ループ	描画中に直接 <code>setState</code> した／effectの依存に毎回変わる値	<code>setState</code> はイベントやeffectの中で。関数は <code>useCallback</code> でメモ化（§ 7）
<code>map</code> で警告「key を付けて」	繰り返し要素に <code>key</code> がない	<code><tr key={item.id}></code> のように一意なkeyを付ける（§ 2.4）
ボタンを押すと即実行される	<code>onClick={fn(arg)}</code> と書いた	<code>onClick={() => fn(arg)}</code> とアロー関数で包む（§ 8.13 ⚠️）
stateを変えたのに画面が変わらない	直接代入した（ <code>x = 1</code> ）	必ず更新関数 <code>setX(1)</code> を使う（§ 4.2 ⚠️）
通信が401で弾かれる	Cookie未送信／未ログイン	<code>axios.ts</code> の <code>withCredentials: true</code> を確認（§ 12④）。再ログイン
「ログインしてるのに403」	権限不足	サーバー側の権限設定を確認。403はトップへ誘導される（§ 8.7⑭）
<code><Link></code> でページが丸ごと再読み込み	<code><a href></code> を使っている	アプリ内移動は <code><Link to></code> に（§ 14.1）
effectが2回走る	StrictMode（開発時のみ）	正常。クリーンアップを正しく書けば本番は1回（§ 13.2）
CSSが当たらない	module を文字列で書いた等	<code>className={styles["名前"]}</code> （module）と <code>"名前"</code> （global）の使い分け（第3章 § 13）
<code>Cannot read properties of undefined</code>	データ到着前に中身を参照	<code>data?.results</code> や、

17. 演習問題

手を動かすと定着します。**ローカル環境**（壊しても安全）で試してください。

1. **stateの初期値を変える**：`KosuList.tsx` の `useState<number>(1)` (`currentPage`) を `useState<number>(2)` にして、最初から2ページ目が表示されることを確認。終わったら戻す。
2. **propsで色を変える**：`KosuList.tsx` の `<Pagination buttonColor="#0ff" .../>` を `buttonColor="#0f0"` (緑) に変え、ページ送りボタンの色が変わることを確認。`Pagination` 側は1行も触らずに変わる——propsの威力を体感する。
3. **変換関数を読む**：`formatTyoku` (§8.3) に `case 7: return "夜勤";` を追加したら、`tyoku2` が `"7"` のデータがどう表示されるか予想し、確認する。
4. **early returnを観察**：`KosuList.tsx` の `if (loading) return ...` を一時的にコメントアウトすると何が起きるか予想し、確認する（データ到着前に表を描こうとしてエラー or 空表示）。終わったら戻す。
5. **依存配列を理解する**：`useEffect` ② (§8.9) の依存配列から `currentPage` を消すと、ページ送りしてもデータが変わらなくなることを確認（なぜかを §5.2 で説明できるように）。必ず戻す。
6. **map と key**：`<tr key={item.id}>` の `key={item.id}` を消すと、Consoleにどんな警告が出るか確認する (§2.4)。戻す。
7. **Linkとaの違い**：`<Link to="/kosu-menu">` を `<a href="/kosu-menu">` に変えて、移動時に画面が一瞬白くちらつく（再読み込みされる）ことを観察する (§14.1)。戻す。
8. **新しいコンポーネントを作る**：`Components/Hello.tsx` を作り、`const Hello: React.FC<{name: string}> = ({name}) => <p>こんにちは {name} さん</p>;` を書いて `export default`。`KosuList` で `<Hello name="工数" />` と置いて表示を確認する（コンポーネント作成・props・JSXの総復習）。

18. この章のまとめ

- Reactは「**データを変えたら、画面は設計図から自動で作り直される**」宣言的なライブラリ。**仮想DOM**で差分だけを高速に反映する (§0)。

- 画面は **コンポーネント（部品）** の組み立て。1関数=1部品で、JSXを返す。名前は **大文字始まり**（§1）。
- **JSX** はJS内にHTMLを書く記法。 `class→className` ・ `for→htmlFor` ・ `{ }` でJS埋め込み。条件は `&&` /三項演算子、繰り返しは `map`（要 `key`）、まとめ役は `<>` `</>`（§2）。
- **props** は親→子の一方通行の設定値。子の変更は「親から渡された関数を呼ぶ」（§3）。
`Pagination` が好例。
- **state（useState）** は変化する値。`setX` で更新すると **再描画**。直接代入は禁止（§4）。
- **useEffect** は通信などの副作用を、**依存配列** のタイミングで実行。仕掛けたものは **クリーンアップ** で外す（§5）。
- **useRef** は再描画を起こさない箱。値の保持とDOM参照に使う（§6）。** `useCallback` ** は関数を使い回し、effectの無限ループを防ぐ（§7）。
- **KosuList.tsx 全215行**（import／interface／変換関数／state・ref／fetchData／2つのeffect／handleSearch／早期リターン／JSX）を1行ずつ読み切った（§8）。
- ** `Pagination.tsx`（**propsだけで動く再利用部品**）、`Loading.tsx`（**クリーンアップ**）、`TableContainer.tsx` **（children・resize対応）を全解説（§9～11）。
- **axios.ts 全8行**：baseURL（接続先）・withCredentials（ログイン維持）・CSRF（なりすまし防止）の共通通信設定（§12）。
- ** `index.tsx` **： `Router` / `Routes` / `Route` / `:id` でURLと画面を対応づける **SPA** のルーティング。アプリ内移動は ** `<Link>` **、処理判断の移動は ** `useNavigate` **（§13・§14）。
- 本質は「**state→画面（下り：props）**」と「**操作→state更新（上り：渡された関数）**」の循環（§15）。

次は、画面の裏でデータを処理するサーバー側の言語、「[06_Python基礎.md](#)」へ進みます。ここまでで「画面（フロントエンド）」の全体像が手に入りました。次章からは、その画面に応答する「サーバー（バックエンド）」の世界に入ります。